Highlights

FOLoRA: Fisher-Orthogonalized Low-Rank Adaptation for Continual Learning

- FOLoRA protects the Fisher-important directions of a shared LoRA adapter.
- Fisher weighting sets the operating strength, not the accuracy ceiling.
- FOLoRA minimizes the leading term of a second-order bound with explicit remainder.
- One setting transfers across benchmarks; EWC-LoRA's default strength does not.
- Beats L2P, CODA-Prompt, O-LoRA and InfLoRA on both benchmarks; no module per task.

FOLoRA: Fisher-Orthogonalized Low-Rank Adaptation for Continual Learning

Jiang Tao[a,*]

[a]*School of Intelligent Information Engineering, Heilongjiang University of Finance and Economics, Harbin, China*

Abstract

Parameter-efficient fine-tuning (PEFT) methods such as low-rank adaptation (LoRA) freeze a pre-trained backbone and train few parameters, suiting continual learning on resource-constrained devices. However, sequentially fine-tuning a shared LoRA adapter still forgets catastrophically: the low-rank subspace is overwritten by new tasks. We propose **FOLoRA** (Fisher-Orthogonalized Low-Rank Adaptation), a replay-free, fixed-budget method protecting the directions past tasks deem important. FOLoRA accumulates, per adapted layer, a second-order importance kernel from the empirical Fisher on adapter parameters. Its top eigendirections form an *importance-weighted protected subspace*, and a regularizer penalizes alignment of the adapter displacement with them, weighted by eigenvalue. We further derive a second-order forgetting bound with an explicit remainder and show FOLoRA's regularizer minimizes its leading term, using it as a design principle rather than a numerical predictor. Under a common frozen-feature nearest-class-mean decision rule, FOLoRA shows no significant difference from the strongest

*Corresponding author

Email address: jiangtao.ml@foxmail.com (Jiang Tao)

tuned EWC-LoRA configuration on either benchmark; its positive result is against the parameter-efficient family, where it improves significantly over L2P, CODA-Prompt, O-LoRA and InfLoRA on both benchmarks while allocating no module per task, so its footprint stays constant in the task count ($\sim 18\times$ smaller than a per-task adapter bank), and it needs no replay, adds no inference cost and merges into the backbone. One setting is used unchanged on both benchmarks, where EWC-LoRA's pre-registered default strength does not transfer. Only the classifier head grows with the task count, shared by all methods and discarded at evaluation. The trade-off is a larger importance-state footprint than EWC's diagonal approximation.
Keywords: continual learning, catastrophic forgetting, parameter-efficient fine-tuning, low-rank adaptation, Fisher information

1. Introduction

Continual learning aims to learn a sequence of tasks without catastrophically forgetting previously acquired knowledge [1]. The dominant strategies — rehearsal [2], regularization [3, 4], and dynamic architectures [5] — were largely developed in the regime of training a model from scratch. Recently, however, the community has shifted toward *pre-trained models* (e.g., Vision Transformers) combined with *parameter-efficient fine-tuning* (PEFT), in which the backbone is frozen and only a tiny set of parameters is learned per task [6, 7, 8]. This shift has produced a fast-moving literature on prompt-, adapter-, and subspace-based PEFT for continual learning [9, 10, 11, 12, 13].

Low-rank adaptation (LoRA) [14] is a particularly attractive PEFT primitive for continual learning, and recent variants exploit it in different ways [15,

16, 17, 18]: a shared LoRA adapter keeps *its own* trainable parameter count fixed regardless of the number of tasks, which is ideal for resource-constrained deployment. However, sequentially fine-tuning a shared adapter still suffers from catastrophic forgetting — although the number of trainable parameters is small, the low-rank subspace is repeatedly overwritten, and the representational structure learned for earlier tasks is destroyed.

A line of recent work addresses this by allocating *orthogonal* low-rank subspaces to different tasks [8, 19], and refined task-wise allocation schemes continue to appear [20, 21, 22]. These methods prevent interference by making the new task's adapter (nearly) orthogonal to those of previous tasks. While effective, they exhibit two limitations. First, they protect all directions of the past subspace *equally*, ignoring the fact that different directions carry vastly different amounts of information about a task. Second, most of them store a *new* adapter per task, so the retained parameter count grows linearly with the number of tasks — even though, for O-LoRA, only the current adapter is trained at any one time, so the growth is in stored rather than in trained parameters.

In this paper we propose **FOLoRA** (Fisher-Orthogonalized Low-Rank Adaptation), a replay-free, fixed-budget method that suppresses the forgetting of unregularized LoRA by protecting past tasks' *important* directions in proportion to their importance, at accuracy parity with the strongest tuned regularization baseline and with the regularization strength transferring across benchmarks. Our contributions are:

1. We frame forgetting in low-rank continual learning through a second-order lens and derive an upper bound that decomposes forgetting into

Fisher-weighted subspace overlaps. We audit the slack of that bound empirically rather than assuming it away, and use it as a design principle for what to penalize rather than as a numerical predictor of how much forgetting a given setting removes (Sec. 4). The version summed over all past tasks carries a first-order correction for historical drift, whose size we assume rather than bound; Remark 1 states that assumption explicitly.

2. We propose FOLoRA, which estimates a per-layer second-order importance kernel from the empirical Fisher information restricted to the adapter parameters, forms an importance-weighted protected subspace from its top eigendirections, and regularizes the adapter to stay orthogonal to these directions (Sec. 3).
3. We show that FOLoRA's regularizer is exactly the Lagrangian penalty for constraining the leading term of a second-order forgetting bound, realizing a principled stability–plasticity trade-off, and that it contains equal-weight orthogonalization — the geometry underlying O-LoRA-style subspace methods — as the special case in which all protected directions carry equal weight.
4. On Split CIFAR-100 and Split ImageNet-R, FOLoRA matches the strongest regularized baseline in average accuracy with no replay buffer and no growth in parameters, while improving significantly over the parameter-efficient prompt and subspace baselines. Its main configuration is selected once and carried across both benchmarks unchanged, whereas EWC-LoRA's pre-registered default strength is not the best point of its own grid on either benchmark: at λ=100 it sits 0.8 points

below its tuned optimum on CIFAR-100 — the weakest of its four ten-seed points — and 2.8 points below it on ImageNet-R. We also quantify the importance-model memory FOLoRA pays for its rank-k subspace representation relative to EWC's diagonal one, and we report the run-to-run reproducibility floor that sets the resolution of these comparisons (Sec. 5).

2. Related Work

Continual learning. Classical continual learning methods fall into three families. Rehearsal methods store and replay a subset of past data [2]; regularization methods add a penalty that slows down learning on parameters important to past tasks, most notably EWC via the diagonal Fisher information [3] and SI/MAS via synaptic importance [4, 23]; dynamic methods grow the network per task [5, 24]. Recent surveys of the field appear in [13, 12, 25], and the exemplar-free, representation-level, and distillation-based variants most relevant here in [26, 27, 28, 29]. Our method is a regularization method, but it operates on the low-rank subspace of a frozen backbone rather than on raw parameters.

Parameter-efficient continual learning with pre-trained models. A recent and effective line of work keeps a pre-trained backbone frozen and learns only prompts or adapters; the space is surveyed in [9, 10, 11, 30]. Prompt-based methods — L2P [6], DualPrompt [31], and CODA-Prompt [7] — learn a pool of prompts and select or compose them per input. LoRA-based methods attach low-rank adapters. Related designs differ in where they spend the per-task budget: expandable or branched subspaces [20, 16, 17, 21], decoupled

scaling rules [15], orthogonal adapter banks [32], further adapter variants [33, 34, 35, 36], dual-consolidation and hierarchical prompt decompositions [37, 38], and prototype- or prompt-routing schemes [39, 40, 41, 42]; for the online and task-free regime, see [18]. Two recent studies measure what low-rank adaptation costs and saves under a deployment budget rather than under a continual-learning protocol — a systematic comparison of LoRA, QLoRA and full fine-tuning on ConvNeXt-Tiny [43], and a saliency-guided quantized adaptation of tabular-to-text models [44] — and they inform the memory accounting of Sec. 5.6.

Orthogonalization and subspace allocation. A distinct line makes *where* a new task may write the design variable, rather than how strongly. Its classical form is gradient projection: orthogonal gradient descent [45], GPM [46] and the null-space-of-feature-covariance update of Adam-NSCL [47] retain a basis describing past tasks and constrain new updates to its orthogonal complement, a construction that also admits generalization guarantees [48]. The same idea reappears in PEFT as subspace allocation for low-rank adapters: O-LoRA [8] orthogonally initializes each task's adapter against previous tasks' input subspaces, and InfLoRA [19] places the new task's subspace in the intersection of the new task's gradient space and the orthogonal complement of the old tasks' gradient space, removing interference while retaining plasticity. The same axis is now being explored for prompt tuning, via null-space constructions [49].

Two properties are shared across this line, and they are what FOLoRA changes. First, the protected subspace is obtained from a *structural* criterion — the past tasks' input or feature covariance, a gradient subspace, or

an explicit orthogonality constraint — rather than from the curvature of the past loss. Second, within the protected subspace every retained direction is defended equally: the budget decides *how many* directions to keep, never *how much* each one matters. FOLoRA keeps the subspace framing and replaces both choices with a single object, the adapter block of the accumulated empirical Fisher, so that the retained directions are those in which the past loss is most curved and each is penalized in proportion to its curvature. The equal-weight geometry of this family is then the special case $\sigma_j \equiv 1$ of FOLoRA's penalty (Sec. 4), and we compare against O-LoRA and InfLoRA empirically in Sec. 5.2.

Second-order methods and curvature-weighted penalties. EWC [3] uses the diagonal Fisher as a per-parameter importance weight. FOLoRA combines that idea with the subspace framing above: it uses the top-k spectrum of the adapter block of the second-order expansion — not the diagonal Fisher, and not the full $d \times d$ matrix either — and applies it as a soft, importance-weighted orthogonal projection rather than a hard null-space constraint, which avoids the rank saturation that hard projections suffer once the null space shrinks. Recent analyses of flatness-aware optimization approach the same goal from other angles [50], flatness itself has been connected to generalization bounds [51], and gradient-norm regularization acts on the same stability–plasticity axis without a curvature weighting [52].

Fisher information inside low-rank adaptation. The two ingredients FOLoRA combines — empirical Fisher information and low-rank adaptation — have met before, and the differences are worth stating precisely

rather than leaving to the reader. FR-LoRA [53] is the closest in problem setting: it targets continual learning and regularizes LoRA adapters with Fisher information. It differs in *what* is regularized — a per-task adapter, so the trainable budget grows with the sequence exactly as in the branch-per-task family above — and in the role the Fisher plays, a parameter-importance penalty in the spirit of EWC rather than a spectral subspace of the adapter displacement. Three further works use Fisher information for LoRA but not for continual learning: FiLoRA [54] uses it to guide the low-rank update, FI-LoRA [55] to decide how much rank each layer receives dynamically, and the Fisher-subspace construction of Feng et al. [56] to choose an initialization rather than to constrain training — an initialization changes where training begins, whereas FOLoRA's penalty acts at every step. None of the three faces a task sequence, and none of the four penalizes the alignment of a *shared* adapter with an importance-weighted past subspace, which is the object this paper derives and evaluates. The scope of that negative claim is a search for work combining Fisher information with low-rank adaptation, as of this writing; the four works are listed together here so that the novelty claim can be checked rather than taken on trust.

Weight regularization on a shared low-rank adapter. One further work bears directly on our *baseline* rather than on the method, and we position it explicitly. Zheng et al. [57] revisit weight regularization for the same setting we study — one shared low-rank update, a constant trainable budget, an explicit task sequence — and argue that applying EWC to the LoRA factors A and B separately is not equivalent to applying it to the merged update $\Delta W = BA$, estimating importance on the full-dimensional combined update

instead. Our EWC-LoRA baseline is the factor-wise variant, i.e. the formulation Zheng et al. identify as the weaker one. We state this here rather than leaving it for a reader to discover: the parity reported in Sec. 5.2 is parity with a diagonal Fisher on the factorized adapter parameters, and not with a Fisher on the merged update; reproducing their estimator is a different study from this one. Their reported accuracies use a different protocol and are not comparable to ours, and we draw no conclusion from them. The distinction between the two methods is orthogonal to that baseline question: FOLoRA keeps the accumulated Fisher's eigenstructure and weights each protected direction by its eigenvalue, penalizing a shared displacement; the factor-wise EWC-LoRA penalizes every factor entry with the same diagonal weight, and the merged-update formulation of Zheng et al. penalizes directions in the weight space of ΔW without selecting the leading eigendirections of the adapter-space importance.

Theory of forgetting. Second-order analyses of forgetting are well established for models trained from scratch: the forgetting of a task is governed by the curvature of its loss landscape and by the overlap between successive tasks' input distributions or NTKs [58, 59, 60, 61]. We build on this line but instantiate it in the low-rank adapter subspace of a frozen pre-trained model, and we use it constructively: the leading term of the bound is exactly the quantity that a subspace regularizer can minimize, which yields the FOLoRA objective (Sec. 4). Recent theory sharpens this picture for lazy-training and kernel regimes [62, 63], and the empirical counterpart — that low-rank adaptation forgets in a measurably different way from full fine-tuning [64] — is part of what motivates working in the adapter subspace rather than in the

full parameter space.

Low-rank adaptation. LoRA [14] approximates weight updates by low-rank matrices and has become a standard PEFT primitive. Subsequent work studies initialization, rank allocation, and merging [36, 34]. We build directly on LoRA and extend it with a second-order anti-forgetting mechanism specialized for continual learning.

Evaluation protocols in PEFT-based continual learning. Results in this setting are strongly protocol-dependent. Keeping the standard trainable classification head that grows with the task sequence, class-incremental accuracy is typically in the single digits even when task-incremental accuracy is above 90%, because the loss is dominated by the classifier rather than by the adapted features; replacing the head at evaluation time with a nearest-class-mean classifier over frozen features raises the same checkpoints into the 70–80% range. Two consequences matter for how a method should be tuned and compared. First, hyperparameters — the regularization strength in particular — are only identifiable relative to the protocol under which they were selected, and an optimum found under a classifier-dominated metric need not be the optimum under the reported one. Second, a ranking established under one protocol does not transfer to another. Accordingly, all defaults reported in this paper are re-selected under the frozen-feature nearest-class-mean protocol, and every comparison we draw is made within that protocol. That the protocol can dominate the reported conclusion is now being recognized independently: recent work shows the class-incremental average is sensitive to evaluation choices and can misrepresent representation quality [65, 66, 67],

that classifier bias and calibration distort comparisons unless handled explicitly [68, 69], and that simple frozen-feature classifiers over a pre-trained backbone remain competitive with more elaborate designs [70, 71, 72], as does a slow learner with post-hoc classifier alignment [73].

Gap addressed in this work. None of the lines above supplies what this paper sets out to provide. Subspace-based methods [8, 19, 47] protect a past task's subspace as a whole, so every retained direction is defended with the same strength regardless of how much the past loss actually depends on it, and most of them allocate a branch per task, so the trainable parameter count grows with the sequence length. FR-LoRA is the one exception in setting but not in structure [53]: it inherits the same per-task allocation, so what it regularizes is a per-task adapter rather than a shared subspace, and its trainable count still grows with the sequence. Second-order analyses of forgetting [58, 61] characterize *why* forgetting happens but are developed for models trained from scratch and are not instantiated in a fixed low-rank adapter, nor do they yield an objective one can simply train with. What is missing is a regularizer that is *derived* from a forgetting bound carrying an explicit remainder, that weights protected directions by their curvature rather than uniformly, and that keeps the parameter budget constant across tasks. The derivation is used here as a statement of *which* quantity to penalize rather than as a numerical prediction of *how much* forgetting a setting removes: Sec. 4 audits the bound's slack empirically instead of assuming it away, and the method's empirical justification is the ablations in Sec. 5.5. Prior work in this journal addresses adjacent aspects — gradient-based task organization [22], gradient-norm regularization for dissimilar tasks [52], and

task-adaptive routing [39] — but none of it derives the protected directions from a forgetting bound, which is the specific step taken here. FOLoRA is constructed to fill that gap.

3. Method

3.1. Preliminaries

Low-rank adaptation (LoRA). LoRA [14] freezes a pre-trained weight matrix $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and injects a trainable low-rank residual,

$$W \;=\; W_0 \;+\; \Delta W \;=\; W_0 + \frac{\alpha}{r}\, BA, \tag{1}$$

where $A \in \mathbb{R}^{r \times d_{\text{in}}}$, $B \in \mathbb{R}^{d_{\text{out}} \times r}$ and $r \ll \min(d_{\text{in}}, d_{\text{out}})$. The scalar α/r absorbs the rank normalization. The forward pass of an adapted layer is

$$h \;=\; W_0 x + \frac{\alpha}{r}\, BAx \;=\; W_0 x + \delta, \qquad \delta := \frac{\alpha}{r}\, BAx \in \mathbb{R}^{d_{\text{out}}}, \tag{2}$$

where δ denotes the adapter's residual contribution.

Class-incremental continual learning (CIL). We consider a sequence of T tasks $\{\mathcal{T}_t\}_{t=1}^{T}$. Task $\mathcal{T}_t$ provides a training set $\mathcal{D}_t = \{(x_i, y_i)\}$ over c new classes, and the classifier head is grown to cover all classes seen so far. At test time the task identity is not available, so prediction is made over the union of all seen classes. A single shared LoRA adapter is trained sequentially; its parameter count is $r(d_{\text{in}} + d_{\text{out}})$ per layer and is *independent* of T.

3.2. Motivation: forgetting as subspace overlap

Let ξ_τ collect all *trainable* parameters at the end of task $\mathcal{T}_\tau$ — the adapter parameters θ_τ (the flattened low-rank factors of Sec. 3.3) together with the

classifier-head parameters — and let $\delta\xi = \xi - \xi_\tau$ be a later task's displacement of them. A second-order Taylor expansion of task τ's loss around ξ_τ gives

$$\mathcal{L}_\tau(\xi_\tau + \delta\xi) \;\approx\; \mathcal{L}_\tau(\xi_\tau) + \nabla_\xi \mathcal{L}_\tau^\top \delta\xi + \tfrac{1}{2}\,\delta\xi^\top \mathcal{F}_\tau\,\delta\xi, \tag{3}$$

where $\mathcal{F}_\tau = \mathbb{E}_{(x,y)\sim\mathcal{D}_\tau}\big[\nabla_\xi \log p(y|x)\,\nabla_\xi \log p(y|x)^\top\big]$ is the empirical Fisher information matrix of task τ over those parameters. At the end of task τ the parameters are (approximately) stationary for that task's loss, $\nabla_\xi \mathcal{L}_\tau(\xi_\tau) \approx 0$; this is stated as an explicit assumption (Assumption 1) and its residual under the regularizer is quantified in Sec. 4. Hence the *forgetting* of task τ — the increase in its loss caused by subsequent learning — is dominated by the quadratic term $\frac{1}{2}\,\delta\xi^\top \mathcal{F}_\tau\,\delta\xi$. Forgetting is therefore driven by the alignment of the new displacement with the directions in which task τ's loss is sharply curved (i.e., the top eigendirections of $\mathcal{F}_\tau$). Of that quadratic form, FOLoRA acts on the part involving the adapter parameters: its regularizer penalizes alignment of the adapter displacement $\delta\theta$ with the directions in which the adapter block of $\mathcal{F}_\tau$ is sharply curved. Sec. 4 makes this precise, including the head terms that FOLoRA does not constrain.

This observation separates FOLoRA from prior work: O-LoRA [8] treats every direction of the past low-rank subspace as equally important, whereas Eq. (3) shows that directions should be protected *in proportion to their curvature*. Protecting all directions equally over-constrains the adapter and wastes the fixed rank budget on directions that carry little information; weighting by curvature spends that budget where forgetting actually happens. We note that this is an argument about *where the budget is spent*, not a claim that weighting wins: the theory does not predict an accuracy advantage for the weighted variant (Sec. 4.6 states the caveat explicitly), and Sec. 5.5 reports

what our experiments show.

3.3. FOLoRA

We now describe FOLoRA. For each adapted layer, let $\theta \in \mathbb{R}^d$ denote the flattened adapter parameters (the LoRA matrices A and B, so $d = r(d_{\text{in}} + d_{\text{out}})$), and let $g_\theta = \nabla_\theta \log p(y|x)$ denote the gradient of the log-likelihood with respect to these parameters.

Importance estimation. After finishing task $\mathcal{T}_\tau$, we estimate a per-layer *importance kernel* from the per-sample parameter-gradient autocorrelation,

$$F_\tau \;=\; \mathbb{E}_{(x,y)\sim\mathcal{D}_\tau}\big[\, g_\theta(x,y)\, g_\theta(x,y)^\top \,\big] \;\in\; \mathbb{R}^{d\times d}, \tag{4}$$

which is the empirical Fisher information of task τ restricted to the adapter parameters — i.e. the adapter block $\mathcal{F}_\tau^{\theta\theta}$ of the full-parameter Fisher of Sec. 4 — and which we write F_τ for short below. To keep the estimate tractable we never materialize the full $d\times d$ matrix; we collect the per-sample gradients into a matrix (one row per sample) and retain only its top-k singular directions, i.e., a rank-k approximation of F_τ. We accumulate these kernels across tasks,

$$\bar{F} \;=\; \sum_{\tau<t} F_\tau. \tag{5}$$

Eq. (5) is the *object* the protected subspace is meant to represent. The implementation stores, per layer, a rank-k factor rather than the full F_τ, so what is actually accumulated is computed on the concatenation of the retained factors of all previous tasks; this is an incremental-PCA approximation to the sum in Eq. (5) rather than the sum itself, and it is exact only when the retained top-k subspace of the concatenated factors coincides with the union

of the per-task top-k subspaces. We keep the exact form in the equations for readability and use the approximation in all experiments; the two agree in the limit $k \to d$.

Protected subspace. At the start of task $\mathcal{T}_t$, we eigendecompose the accumulated kernel $\bar{F} = \sum_j \sigma_j u_j u_j^\top$, where $\{\sigma_j\}_j$ are the eigenvalues sorted in decreasing order and $\{u_j\}$ the corresponding orthonormal eigenvectors in $\mathbb{R}^d$. We retain the top-k eigendirections as an *importance-weighted protected subspace* $\mathcal{P}_t = \{(\sigma_j, u_j)\}_{j=1}^k$.

Orthogonalization regularizer. Let θ^{t-1} denote the adapter parameter snapshot recorded after task $\mathcal{T}_{t-1}$, and let $\delta\theta = \theta - \theta^{t-1}$ be the parameter displacement accumulated while training task $\mathcal{T}_t$. During the training of task $\mathcal{T}_t$, FOLoRA minimizes

$$\mathcal{L} \;=\; \mathcal{L}_{\mathrm{CE}}(\theta) \;+\; \lambda \underbrace{\sum_{j=1}^{k} \sigma_j \left(u_j^\top \delta\theta\right)^2}_{\Omega(\delta\theta)}, \tag{6}$$

where $\lambda > 0$ controls the strength and each $u_j \in \mathbb{R}^d$ is a protected direction in the same parameter space as $\delta\theta$. The term $(u_j^\top \delta\theta)^2$ penalizes the alignment of the *displacement* $\delta\theta = \theta - \theta^{t-1}$—rather than the absolute θ—with the protected direction u_j, weighted by the importance σ_j. Using the displacement is essential: at the start of task $\mathcal{T}_t$ we have $\delta\theta = 0$, so the regularizer vanishes and the adapter's existing alignment with past tasks is preserved; only *new* movement into protected directions is penalized. Penalizing the absolute θ would instead push the adapter away from the past tasks' directions and actively erase them, defeating the purpose of protection. Since the

regularizer is a differentiable function of A and B, it integrates with standard backpropagation and requires no hard projection, no replay buffer, and no task identity at inference.

Why importance weighting matters. Setting all $\sigma_j = 1$ reduces Ω to an equal-weight orthogonalization, which is the geometry underlying O-LoRA-style orthogonal subspace methods (Sec. 4.6 states the relation exactly). In contrast, FOLoRA's eigenvalue weighting (i) strongly suppresses reuse of high-curvature directions, which is where forgetting is concentrated, and (ii) leaves low-curvature directions nearly free, so the adapter can reuse them for new tasks without wasting rank. This is the mechanism by which FOLoRA spends a fixed rank budget on the directions the bound identifies as costly. We note that the theory does not by itself predict an accuracy advantage for weighting over equal weighting — a larger penalty family is not automatically a better one, and Sec. 4.6 says so explicitly; whether it helps in practice is an empirical question, and Sec. 5.5 reports what our experiments show.

Fig. 1 shows the procedure end to end: the forward pass of one task on the solid row, and the one-time update that produces the penalty for the next task on the dashed row. Two properties of that picture are the ones the rest of the paper relies on: the adapter and its parameter count are shared across all tasks, so nothing in the solid row grows with T; and the quantity that is updated between tasks is the importance state $\{(\sigma_j, u_j)\}$, so the memory FOLoRA spends is an importance-model cost rather than a parameter cost (quantified in Table 7). The same procedure is given in pseudocode as Algorithm 1.

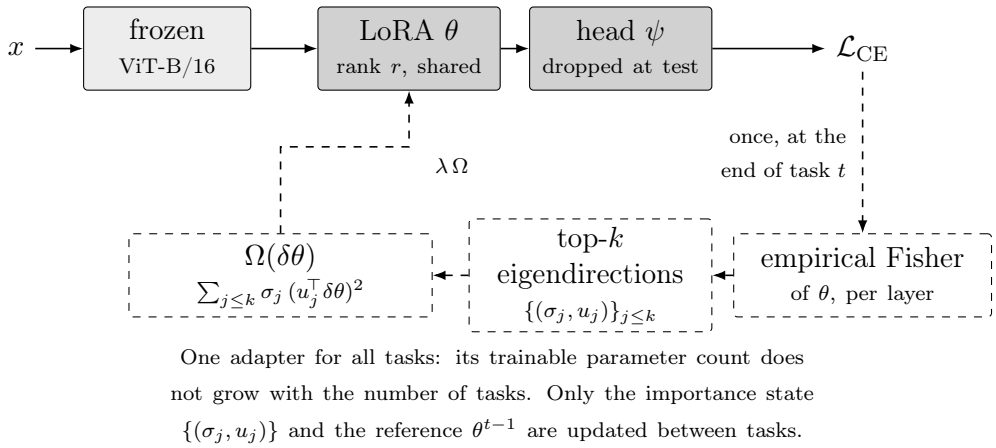


Figure 1: FOLoRA. **Solid row**: the forward pass of task t, where a single shared rank-r adapter θ is trained on a frozen ViT-B/16 backbone with the penalty $\lambda\Omega$ added to the cross-entropy loss; the head ψ is trained but discarded, since prediction is by nearest class mean in the frozen feature space. **Dashed row**: once, after task t ends, the adapter's empirical Fisher is re-estimated per layer, its top k eigendirections are extracted, and they are accumulated into the kernel $\bar{F}$ defining Ω for the next task. The adapter's rank, hence its trainable parameter count, is fixed for the whole sequence; only the importance state grows, so the memory cost is an importance-model cost, not a parameter cost (Table 7).

3.4. Relation to prior methods

- **EWC** [3] penalizes per-parameter quadratic displacement $\sum_i F_{ii}(\theta_i - \theta_i^*)^2$ using the *diagonal* Fisher. FOLoRA instead uses the adapter block of the second-order expansion and applies it as a rank-k subspace projection rather than a pointwise penalty. The distinction is not that one is exact and the other is not — both truncate the curvature, EWC to its diagonal and FOLoRA to its top-k spectrum — but that they discard different things: EWC keeps all d coordinates and drops every correlation among them, whereas FOLoRA keeps the k directions

Algorithm 1 FOLoRA for class-incremental continual learning

Input: frozen backbone; LoRA rank r; strength λ; budget k.
Initialize shared LoRA adapters $\Delta W^{(l)} = \frac{\alpha}{r} B^{(l)} A^{(l)}$ for each adapted layer l.
$\bar{F}^{(l)} \leftarrow \mathbf{0}$ for all l.
for each task $\mathcal{T}_t$, $t = 1, \ldots, T$ **do**
  if $t > 1$: eigendecompose $\bar{F}^{(l)} = \sum_j \sigma_j u_j u_j^\top$, keep top-$k$ $\{(\sigma_j, u_j)\}$.
  Train adapter parameters $\theta^{(l)}$ and the grown classifier head by minimizing $\mathcal{L}_{\mathrm{CE}} + \lambda \sum_l \sum_j \sigma_j \big(u_j^\top \big(\theta^{(l)} - \theta_{t-1}^{(l)}\big)\big)^2$.
  Record adapter snapshot $\theta_t^{(l)} \leftarrow \theta^{(l)}$.
  Estimate $F_t^{(l)} = \mathbb{E}[g_\theta \, g_\theta^\top]$ on $\mathcal{D}_t$; update $\bar{F}^{(l)} \leftarrow \bar{F}^{(l)} + F_t^{(l)}$.
end for
Output: a single shared adapter with fixed parameter budget.

of largest curvature with their correlations intact and drops the rest. Which truncation suits a fixed-rank adapter is an empirical question, answered for this setting by the comparison in Sec. 5.2.

- **O-LoRA** [8] allocates a new adapter per task and initializes it orthogonally to previous tasks' input subspaces, treating all directions equally; FOLoRA keeps a *single* fixed-budget adapter and protects directions in proportion to their Fisher importance.

- **GPM** [46] and related null-space methods project gradients onto a null space; FOLoRA realizes a *soft*, importance-weighted orthogonal projection that avoids the rank saturation of hard projections.

4. A second-order view of forgetting: design motivation

We derive a second-order bound on the forgetting of a previously learned task under sequential low-rank adaptation, with an explicit remainder term, and show that the FOLoRA regularizer minimizes the leading term of that bound. We then relate the bound to the equal-weight orthogonalization used by O-LoRA-style methods.

We present the bound as a *design principle rather than a numerical predictor*. We do not claim it is tight, and we never use it to predict how much forgetting a given λ removes. Sec. 4.4 audits the three quantities the expansion rests on — the cubic remainder, the stationarity residual, and the calibration of the Fisher estimate — and reports that the first two are the same order as the quadratic term rather than negligible, and that the third is not a calibrated surrogate for the measured curvature at the operating point we probe. What the theory does establish, and what the method is built on, is structural: the leading term of the expansion is a curvature-weighted projection of the adapter displacement onto the past tasks' eigendirections, and Eq. (16) shows that FOLoRA's penalty is the Lagrangian form of that quantity. The *empirical* case for the method is made separately, by the λ=0 anchor in Sec. 5.5, and not by the bound.

4.1. Setup and assumptions

Let $\theta_\tau \in \mathbb{R}^d$ denote the *adapter* parameters of one adapted layer after learning task $\mathcal{T}_\tau$ — the entries of that layer's low-rank factors A, B, flattened, so that $d = r(d_{\text{in}} + d_{\text{out}})$ — and let ψ_τ denote the classifier-head parameters at that point. Every adapted layer is treated identically, so we carry the analysis

for a single layer and drop the layer index; the total adapter term is the sum over layers of the per-layer terms derived below. We write $\xi_\tau = (\theta_\tau, \psi_\tau)$ for the full trainable vector. For a later task $\mathcal{T}_t$ ($t > \tau$), let

$$\delta\theta^\tau = \theta_t - \theta_\tau, \qquad \delta\psi^\tau = \psi_t - \psi_\tau, \qquad \delta\xi^\tau = (\delta\theta^\tau, \delta\psi^\tau) = \xi_t - \xi_\tau \tag{7}$$

be the adapter, head and full-parameter displacements accumulated *between* τ and t, respectively. We measure the forgetting of task τ by the increase of its empirical loss,

$$\Delta\mathcal{L}_\tau \;=\; \mathcal{L}_\tau(\xi_t) - \mathcal{L}_\tau(\xi_\tau). \tag{8}$$

Our analysis is built on the following assumptions, which we state explicitly because each one is an approximation that the bound inherits.

Assumption 1 (Task stationarity)**.** *ξ_τ is a stationary point of the* unregularized *task loss, $\nabla_\xi \mathcal{L}_\tau(\xi_\tau) = 0$.*

Assumption 2 (Hessian regularity)**.** *$\mathcal{L}_\tau$ is twice continuously differentiable on the segment $[\xi_\tau, \xi_t]$, and its Hessian is Lipschitz there with constant L_3: $\|H_\tau(\xi) - H_\tau(\xi_\tau)\|_2 \le L_3\|\xi - \xi_\tau\|$.*

Assumption 3 (Fisher approximation)**.** *The Hessian at the stationary point is approximated by the empirical Fisher, $H_\tau(\xi_\tau) \approx \mathcal{F}_\tau$,* assumed *rather than derived, following the standard practice of the EWC family [3, 4]. We state it as an assumption because the Gauss–Newton equivalence holds for the* true *Fisher at a maximum-likelihood point under a correctly specified model [74], whereas the empirical Fisher estimated here from a finite per-sample budget is in general a different matrix and can deviate substantially from the Hessian outside that regime [75]. The consequences of this substitution are*

not absorbed silently: they appear explicitly as ε_{F} in Eq. (9), which is by construction the exact error of the substitution at the given displacement.

Defining ε_{F} as the *exact* error rather than an upper bound on it has a consequence worth stating: Eq. (9) does not actually require Assumption 3 in order to be valid — since $\Delta\mathcal{L}_\tau = \frac{1}{2}\delta\xi^{\tau\top}\mathcal{F}_\tau\delta\xi^\tau + \frac{1}{2}\delta\xi^{\tau\top}(H_\tau(\xi_\tau) - \mathcal{F}_\tau)\delta\xi^\tau +$ remainder, the inequality holds for the empirical Fisher as computed, independently of its quality. The assumption governs the *usefulness* of the bound rather than its validity: it is what licenses treating ε_{F} as small and $\mathcal{F}_\tau$ as an object worth regularizing against.

Assumption 1 is the same assumption used by EWC [3] and GPM [46]. It applies to the *unregularized* task loss: FOLoRA's training objective is $\mathcal{L}_{\mathrm{CE}} + \lambda\Omega$, so at the end of task $\mathcal{T}_\tau$ we have $\nabla_\xi\mathcal{L}_{\mathrm{CE}}(\xi_\tau) = -\lambda\nabla_\xi\Omega(\theta_\tau)$ rather than exactly zero. We discuss the size of this deviation in Remark 2 and quantify it empirically in Sec. 5.6.

4.2. A second-order forgetting bound

Theorem 1 (Second-order forgetting bound)**.** *Under Assumptions 1–2, the forgetting of task $\mathcal{T}_\tau$ after learning task $\mathcal{T}_t$ satisfies*

$$\Delta\mathcal{L}_\tau \;\le\; \frac{1}{2}\,\delta\xi^{\tau\top}\mathcal{F}_\tau\,\delta\xi^\tau \;+\; \frac{L_3}{6}\big\|\delta\xi^\tau\big\|^3 \;+\; \varepsilon_{\mathrm{F}}, \tag{9}$$

where $\varepsilon_{\mathrm{F}} = \frac{1}{2}\big|\delta\xi^{\tau\top}(H_\tau(\xi_\tau) - \mathcal{F}_\tau)\delta\xi^\tau\big|$ is the absolute error incurred by replacing the Hessian with the Fisher (Assumption 3), and $\mathcal{F}_\tau = \mathbb{E}_{(x,y)\sim\mathcal{D}_\tau}[\,g_\xi g_\xi^\top]$, with $g_\xi \equiv \nabla_\xi \log p_\xi(y|x)$, is the empirical Fisher of task τ over all trainable parameters.

Proof. Taylor-expanding $\mathcal{L}_\tau$ at ξ_τ along the displacement $\delta\xi = \xi_t - \xi_\tau$ with the integral form of the remainder gives

$$\mathcal{L}_\tau(\xi_t) = \mathcal{L}_\tau(\xi_\tau) + \nabla\mathcal{L}_\tau(\xi_\tau)^\top \delta\xi + \int_0^1 (1-s)\, \delta\xi^\top H_\tau(\xi_\tau + s\,\delta\xi)\, \delta\xi \,\mathrm{d}s. \quad (10)$$

By Assumption 1 the linear term vanishes. Decomposing the integral,

$$\Delta\mathcal{L}_\tau = \frac{1}{2}\, \delta\xi^\top H_\tau(\xi_\tau)\, \delta\xi + \int_0^1 (1-s)\, \delta\xi^\top \big[H_\tau(\xi_\tau + s\delta\xi) - H_\tau(\xi_\tau)\big] \delta\xi \,\mathrm{d}s. \quad (11)$$

The integrand is bounded by $(1-s)\,\|\delta\xi\|^2\,\|H_\tau(\xi_\tau+s\delta\xi) - H_\tau(\xi_\tau)\|_2 \le (1-s)\,s\,L_3\|\delta\xi\|^3$ by Assumption 2, and $\int_0^1 (1-s)s\,\mathrm{d}s = 1/6$, which bounds the remainder by $\frac{L_3}{6}\|\delta\xi\|^3$. Substituting the Fisher approximation (Assumption 3) into the leading term yields Eq. (9). □

Isolating the adapter block. Because $\xi = (\theta, \psi)$ is a concatenation, the leading term of Eq. (9) splits into an adapter block, a head block and a cross term. Writing $\mathcal{F}_\tau$ in the same block structure,

$$\tfrac{1}{2}\,\delta\xi^{\tau\top}\mathcal{F}_\tau\,\delta\xi^\tau = \underbrace{\tfrac{1}{2}\,\delta\theta^{\tau\top} F_\tau\,\delta\theta^\tau}_{\text{adapter block}} + \underbrace{\delta\theta^{\tau\top}\mathcal{F}_\tau^{\theta\psi}\delta\psi^\tau + \tfrac{1}{2}\,\delta\psi^{\tau\top}\mathcal{F}_\tau^{\psi\psi}\delta\psi^\tau}_{\text{head / cross terms}}, \quad F_\tau := \mathcal{F}_\tau^{\theta\theta}, \quad (12)$$

so that Eq. (9) becomes

$$\Delta\mathcal{L}_\tau \;\le\; \underbrace{\frac{1}{2}\,\delta\theta^{\tau\top} F_\tau\,\delta\theta^\tau}_{\text{adapter term}} + \underbrace{\frac{L_3}{6}\big\|\delta\xi^\tau\big\|^3}_{\text{higher order}} + \varepsilon_{\mathrm{F}} + R_{\mathrm{head}}, \quad (13)$$

where R_{head} collects the two head/cross terms of Eq. (12) and F_τ is the adapter block of $\mathcal{F}_\tau$ — the empirical Fisher restricted to the adapter parameters of one layer, of size $d \times d$. FOLoRA constrains the first term of Eq. (13) and nothing else; we carry R_{head} explicitly rather than dropping it, because

the classifier head is trained between τ and t and its contribution to $\Delta\mathcal{L}_\tau$ is not zero.

Three approximations are made in *estimating* F_τ, as opposed to in the bound itself, and we state all three. First, the estimate is a rank-k truncation, as described in Sec. 3. Second, it is computed per adapted layer and treated as block-diagonal across layers: correlations between the parameters of different adapted layers are not modelled, so the $d \times d$ block above is one diagonal block of the full adapter Fisher of dimension $L\,r(d_{\text{in}} + d_{\text{out}})$ for L adapted layers. Third, the weights are on the scale of an *unnormalized* estimate. The per-direction weight σ_j^2 carried by the released implementation is an eigenvalue of $G^\top G$ for the matrix G of per-sample adapter gradients accumulated over the Fisher budget, and therefore grows with the number of samples in that budget (300 at our default), whereas the diagonal Fisher of the EWC baseline we compare against is explicitly averaged over its samples. A numerical value of λ is consequently meaningful only relative to the convention its own method uses: the λ grids of Sec. 5 are read in that sense, and we do not compare λ across the two families on an absolute scale. All three are properties of the estimator; the bound is stated for the exact F_τ, so none weakens Theorem 1, but all three are part of what a reader comparing the paper to the released code should expect to find. The decomposition is useful only if the adapter term dominates, which holds when the displacement is small. We do not obtain that smallness from the parameterization: a LoRA adapter of rank $r = 16$ per layer is a matrix of fixed rank but of unbounded Frobenius norm, so a bound on rank is not a bound on displacement, and the unprotected directions of $\delta\theta$ carry no constraint at all. What we can say is

narrower — the displacement is penalized directly at every task, so the small-displacement regime is the one the method is designed to operate in rather than one it is guaranteed to stay in, and the drift correction in the remark below is where that gap shows up quantitatively. Whether the higher-order term is in fact negligible at that displacement is a question about the data, not about the algebra, and we measure it rather than assume it: Sec. 4.4 finds it to be the *same order* as the quadratic term, which is why Eq. (13) is described throughout as an upper bound with quantified slack rather than as a tight characterization of forgetting.

Eq. (13) is a bound on the *loss* of task τ, and in that bound the classifier head contributes through R_{head}. Our evaluation, however, does not use the head: under the NCM protocol (Sec. 5.1) the head is discarded at test time and prediction is made by nearest class mean in the frozen feature space, so the adapter is the *only* pathway by which a task's test accuracy can be affected after it has been learned. The bound's scope and the evaluation protocol therefore coincide in the sense that matters for the reported metric, even though they differ in the loss that the bound is stated for.

Remark 1 (Scope: one reference point, and the cross term it introduces)**.** *Summing Eq.* (13) *over all previous tasks* $\tau < t$ *requires a displacement* $\delta\theta^\tau = \theta_t - \theta^\tau$ per *task* τ*, i.e. a snapshot of the adapter parameters at every past task. FOLoRA stores a single reference* θ^{t-1}*, following the same single-reference approximation as EWC [3], and substitutes* $\delta\theta = \theta_t - \theta^{t-1}$ *for it. The substitution is not free, and the correction it requires is* first *order rather than second: writing* $e_\tau = \theta^{t-1} - \theta^\tau$ *for the historical drift, so that*

$\delta\theta^{\tau} = \delta\theta + e_{\tau}$, *the quadratic term of task* τ *becomes*

$$\delta\theta^{\tau\top} F_{\tau}\, \delta\theta^{\tau} = \delta\theta^{\top} F_{\tau}\, \delta\theta \;+\; 2\, \delta\theta^{\top} F_{\tau} e_{\tau} \;+\; e_{\tau}^{\top} F_{\tau} e_{\tau}. \tag{14}$$

The middle term is linear in e_{τ}*. It cannot be absorbed into a* $\mathcal{O}(\|e_{\tau}\|^2)$ *remainder, and an earlier version of this section did so incorrectly. Retaining it explicitly, with* R *the head terms* R_{head} *summed over* τ*, the bound we actually minimize is*

$$\begin{aligned} \sum_{\tau<t} \Delta\mathcal{L}_{\tau} \;\le\; \frac{1}{2}\, \delta\theta^{\top} \Big(\sum_{\tau<t} F_{\tau} \Big) \delta\theta \;+\; \sum_{\tau<t} \delta\theta^{\top} F_{\tau} e_{\tau} \;+\; \frac{1}{2} \sum_{\tau<t} e_{\tau}^{\top} F_{\tau} e_{\tau} \\ +\; (t-1) \Big(\frac{L_3}{6} \|\delta\xi^{\max}\|^3 + \varepsilon_{\mathrm{F}} \Big) \;+\; R, \end{aligned} \tag{15}$$

where we identify $\sum_{\tau<t} F_{\tau}$ *with the accumulated importance kernel* $\bar{F}$ *of Eq.* (5)*. Each cross term is controlled by Cauchy–Schwarz in the Fisher metric,* $|\delta\theta^{\top} F_{\tau} e_{\tau}| \le \|F_{\tau}^{1/2} \delta\theta\|\, \|F_{\tau}^{1/2} e_{\tau}\|$*, so the family is dominated by the leading term whenever the drift is small relative to the current displacement. Formally, if there is a* ρ *with* $\|F_{\tau}^{1/2} e_{\tau}\| \le \rho\, \|F_{\tau}^{1/2} \delta\theta\|$ *for every* $\tau < t$*, then Eq.* (15) *is bounded by* $\big(\frac{1}{2} + \rho + \frac{\rho^2}{2}\big)\, \delta\theta^{\top} \bar{F}\, \delta\theta$ *plus the same remainder.* We have no a priori bound on ρ*: it is set by how far the adapter moved between* τ *and* $t-1$*, which is precisely the quantity the regularizer is meant to keep small, so the assumption is not vacuous but neither is it proved. The scope of the theorem is therefore "forgetting relative to the most recent snapshot, plus a drift correction whose size is assumed," and the statement degrades gracefully rather than being exact for all historical tasks. A bound that holds without the assumption needs one snapshot per task, which is the storage cost FOLoRA is designed to avoid.*

Remark 2 (Deviation from stationarity under the regularizer)**.** *Because FOLoRA minimizes* $\mathcal{L}_{CE} + \lambda\Omega$*, the stationarity Assumption 1 holds only approximately, with residual gradient* $-\lambda\nabla_\xi\Omega(\theta_\tau)$ *— nonzero only in the adapter block, since* Ω *does not depend on the head. Writing* $\Omega(\delta\theta) = \sum_j \sigma_j(u_j^\top\delta\theta)^2$ *gives* $\nabla_\theta\Omega(\theta_\tau) = 2\bar{G}^\top\bar{G}\,\delta\theta$ *where* $\bar{G}$ *stacks the accumulated Fisher directions. The residual therefore vanishes proportionally to the displacement itself: in directions where protection is effective (*$\delta\theta \to 0$*) the assumption is self-consistently satisfied, while in weakly protected directions it degrades linearly in* λ*. This provides a theoretical counterweight to increasing* λ *beyond the range where the displacement is small, and is consistent with the ablation in Sec. 5.5: at* k=16*, where the penalty acts on fewer directions, average accuracy peaks at* $\lambda \in \{3, 10\}$ *and then declines through* $\lambda = 100$ *before saturating by* $\lambda = 300$*. At* k=64 *the same decline is not visible over the range we probed, and we do not claim it there.*

4.3. FOLoRA minimizes the bound

Using the eigendecomposition $\bar{F} = \sum_j \sigma_j u_j u_j^\top$, the leading term of Eq. (15) becomes

$$\frac{1}{2}\,\delta\theta^\top \bar{F}\,\delta\theta \;=\; \frac{1}{2}\sum_j \sigma_j \left(u_j^\top\delta\theta\right)^2, \tag{16}$$

where $\delta\theta \in \mathbb{R}^d$ is the adapter-parameter displacement and each $u_j \in \mathbb{R}^d$ is a unit eigenvector of $\bar{F}$, so every inner product above is between vectors of the same dimension. Equation (16) shows that the dominant term of the forgetting bound is a sum of squared projections of the displacement onto the eigendirections of the accumulated Fisher, weighted by their importance σ_j. Here σ_j denotes an eigenvalue of $\bar{F}$; the implementation stores each

direction scaled by the corresponding singular value $s_j = \sqrt{\sigma_j}$ of the per-sample gradient matrix, so that the squared norm it penalizes carries weight σ_j exactly as written. The FOLoRA regularizer in Eq. (6),

$$\Omega(\delta\theta) = \sum_j \sigma_j \left(u_j^\top \delta\theta\right)^2, \qquad \delta\theta = \theta - \theta^{t-1}, \tag{17}$$

is exactly the penalty term of the Lagrangian relaxation of the constraint that the leading term of the bound remain below a tolerance $\epsilon > 0$,

$$\delta\theta^\top \bar{F}\, \delta\theta \;\leq\; \epsilon, \tag{18}$$

since relaxing that constraint with multiplier λ returns $\mathcal{L}_{\mathrm{CE}} + \lambda\Omega$. Minimizing Ω therefore drives $|u_j^\top \delta\theta|$ toward zero for the largest σ_j, which in turn minimizes the dominant terms of Eq. (16). We therefore interpret FOLoRA as the algorithm that *directly minimizes the leading term of a second-order bound on catastrophic forgetting*, with λ controlling the stability–plasticity trade-off and k controlling the rank of the protected subspace.

4.4. How tight is the bound?

Theorem 1 is an upper bound, and everything above uses only its *structure*. We do not assume it is tight, and we never use it to predict how much forgetting a given λ removes. To make that a measured statement rather than a disclaimer, we probed the bound's own terms on a trained run; the probe protocol, the decision rule frozen before the numbers were computed, and the full measurements are in Appendix B. Three of the approximations the expansion inherits are violated at the operating point we probe:

(i) The remainder is not negligible. The cubic coefficient is of the same order as, and at its worst boundary 2.5×, the quadratic term it is meant to

correct. *(ii) The stationarity residual is not small either.* It does not decay fast enough to be dropped, so the "quadratic plus corrections" reading of the expansion is not numerically available at this λ. *(iii) The finite-sample Fisher estimate is not a calibrated surrogate for the curvature.* It underestimates the measured curvature along the realized displacement by four to five orders of magnitude — which does *not* contradict Theorem 1, stated as it is for the exact F_τ, but does mean that Assumption 3 is far from innocuous here, which is precisely the regime the empirical-Fisher literature warns about [75]. The operative reading is the same in all three cases: the bound's leading term is not a quantitative predictor of forgetting in this setting.

(iv) The same decomposition at the main configuration, and on the harder benchmark. The three findings above come from the probe of Appendix B; we repeated the measurement at the configuration the paper actually uses (λ=3, k=64) on both benchmarks, over the nineteen task boundaries of each run. Two things hold. First, a quadratic-only fit is an excellent description of the loss displacement ($R^2 \geq 0.978$ at every boundary of every seed; median at least 0.9979 on CIFAR-100 and 0.9995 on ImageNet-R), so the second-order term is the right *object* to penalize. Second, the terms the derivation would like to drop are not small: the first-order term is 40–63% of the quadratic term at the median CIFAR-100 boundary and 48% on ImageNet-R, and the cubic remainder exceeds 30% of the quadratic term at 6, 6 and 8 of the nineteen boundaries on the three CIFAR-100 seeds and at 3 of 19 on ImageNet-R, exceeding 100% at four, two and three boundaries respectively. The correction of Sec. 4.2 is therefore not a technicality: at the operating point, the cross term and the remainder carry as much of the expansion as

the leading term does, which is exactly why we state the bound with them displayed and with ρ left as an unverified assumption rather than absorbed.

What this licenses. The bound is used here as a *design principle*. It identifies which quantity a regularizer should suppress — the curvature-weighted projection of the adapter displacement onto the past tasks' protected directions — and Eq. (16) shows that FOLoRA's penalty is exactly the Lagrangian form of that quantity. We make no claim that minimizing it produces a predictable reduction in forgetting, and Sec. 5 does not test such a claim. The empirical case for FOLoRA is made by ablation instead, and it is a clean one: removing Ω from the objective and changing nothing else takes Split CIFAR-100 accuracy from 78.32 to 69.59 (n=5, paired $p = 0.0005$, negative in all five seeds), i.e. down to the level of unprotected sequential fine-tuning (69.31, $p = 0.53$). The entire effect of the method is carried by the term the bound identifies, which is the sense in which the derivation is load-bearing — as motivation for *what* to penalize, not as a guarantee of *how much* it helps.

4.5. Relation to prior forgetting bounds

Second-order analyses of catastrophic forgetting are well established for models trained from scratch. Nguyen et al. [58] show that forgetting is driven by the alignment between successive tasks' loss landscapes and derive a worst-case bound in terms of the curvature of the old task; Evron et al. [59] give tight bounds for linear networks showing that forgetting is governed by the overlap between the old task's input distribution and the directions in which the new task changes the weights; Karakida and Akaho [60] analyse learning curves for continual learning using NTK overlap matrices; and Doan et al. [61]

quantify forgetting through the NTK overlap between tasks. Closer in spirit, orthogonal-gradient descent [48, 45] admits generalization bounds obtained by constraining updates to the null space of previous task gradients.

Our analysis differs from this line in where it is instantiated. Prior results bound forgetting for *full* parameter updates of models trained from scratch, and are used mainly to explain forgetting. We instantiate the bound in the *low-rank adapter subspace* of a frozen pre-trained backbone, where the displacement is supported on a $r(d_{\text{in}} + d_{\text{out}})$-dimensional subspace per layer, and we use the bound constructively: its dominant term is exactly the quantity a subspace-regularizer can minimize, which yields the FOLoRA objective. The resulting penalty differs from null-space projection [46, 48] in being importance-weighted and soft rather than hard, so that the fixed rank budget is spent on the directions with the largest curvature.

4.6. Connection to equal-weight orthogonalization

If all eigenvalues are set to $\sigma_j = 1$, the leading term of the bound, Eq. (16), becomes $\frac{1}{2}\|\bar{U}^\top \delta\theta\|^2$, where $\bar{U} = [u_1, \ldots, u_k]$ is the orthonormal basis of the retained protected subspace — the equal-weight geometry underlying O-LoRA-style orthogonal subspace methods. The eigenvalue-weighted family therefore contains the equal-weight geometry as the special case $\sigma_j \equiv 1$, and the two coincide exactly when the spectrum is flat over the protected subspace; on our checkpoints it is not flat, so the weighted family is the strictly larger one as a family of penalties.

A larger family is not by itself an accuracy claim, and one tempting overclaim is worth pre-empting here: because every coefficient in Eq. (16) is non-negative, weighting by σ_j does *not* reduce the value of the bound at a

fixed displacement – the weighted coefficients can exceed 1. The claim is the weaker and well-defined one that the weighted penalty attains a *lower value of the bound after both objectives are optimized* for a given protected rank k, by spending the fixed rank budget where the curvature is largest. Whether that is realized in practice is an empirical question; Sec. 5.5 characterizes the weighted family over λ and k, and the equal-weight special case $\sigma_j \equiv 1$ is its natural control.

5. Experiments

5.1. Setup

We use a ViT-B/16 backbone pre-trained on ImageNet-1k, frozen throughout, with LoRA adapters (rank $r = 16$, $\alpha = 16$) attached to the MLP down-projection of every transformer block (12 adapters in total). The classifier head is a single linear layer grown as new classes arrive. All methods share the identical backbone and head; they differ only in the adapter regularization.

Datasets. Split CIFAR-100 (20 tasks $\times$ 5 classes) and Split ImageNet-R (20 tasks $\times$ 10 classes), both under the class-incremental protocol (task identity unavailable at test time).

Baselines. (i) *Seq-LoRA*: sequential fine-tuning of the shared adapter with no protection (lower bound); (ii) *EWC-LoRA*: diagonal Fisher on the LoRA parameters [3]; (iii) *O-LoRA*: per-task adapters with orthogonal initialization [8]; (iv) *L2P*: a pool of prompts where the prompt is selected per input by key–query matching [6]; (v) *CODA-Prompt*: a set of prompt components combined by attention [7]; (vi) *InfLoRA*: a per-task subspace

projected to interfere minimally with the previous ones [19]; (vii) *FOLoRA* (ours).

Training. AdamW with learning rate 10^{-3}, batch size 32, and bf16 mixed precision. We train the adapter-based methods (Seq-LoRA, EWC-LoRA, O-LoRA, InfLoRA, FOLoRA) for 5 epochs per task — for the Fisher-weighted ones this avoids over-fitting and keeps the empirical Fisher meaningful (Sec. 5.6), and it places the whole adapter family on one budget — the prompt-based methods (L2P, CODA-Prompt) are trained for 20 epochs per task, their optimal setting since their key-based prompt selection benefits from more epochs. Prompt geometry is a substantive choice rather than a detail — CODA-Prompt's contribution is a *large* pool of components combined by attention — so we use each prompt method's own geometry (L2P: pool 20, length 5; CODA-Prompt: pool 100, length 8) and report CODA-Prompt at its faithful geometry. We also ran CODA-Prompt with L2P's geometry (pool 20, length 5) at 20 epochs; it scores 75.83 on CIFAR-100 against the faithful 74.71. We report the faithful configuration because it is the geometry CODA-Prompt specifies. We state plainly that this is also the configuration in which CODA-Prompt scores the *lower* of the two numbers we measured, so FOLoRA's margin against it is correspondingly larger than it would be against the alternative 75.83; the choice is justified by fidelity to the method, not by the margin it produces. For FOLoRA we fix the main configuration by a grid search and then do not retune it: $\lambda \in \{1, 3, 10\}$ at k=64 and $k \in \{16, 64, 128\}$ at λ=3, plus λ=0 (penalty removed, everything else unchanged) as the ablation control. The main configuration k=64, λ=3 was fixed *before* the extra seeds of the top-up landed and is reported as such

throughout; λ=10 is not distinguishable from it on the five seeds the two share ($\Delta = +0.08$, $p = 0.887$, Table 6) and appears as a grid point there, not as the main configuration. The importance kernel is estimated from up to 300 per-sample gradients per task. EWC's regularization strength is likewise tuned on CIFAR-100: $\lambda \in \{1, 3, 10, 30\}$ at five seeds, and a wider grid $\lambda \in \{100, 300, 1000, 3000\}$ at ten seeds that was run on both benchmarks (λ=3000 was not run on ImageNet-R). In Table 1 we report each benchmark's *best-scoring* EWC point from that grid — λ=300 on CIFAR-100 and λ=1000 on ImageNet-R — i.e. we deliberately take the strongest configuration of the baseline *on each benchmark*, and we report the whole curve, including the pre-registered baseline λ=100, in Table 3. Selecting these strengths is itself an optimization problem, and here it is solved exhaustively: every point we compare against is an actual run whose configuration file is in the repository. Two properties of that selection belong here rather than in a footnote. First, it is *test-set* selection: the EWC-LoRA operating point we report on a benchmark is the point of its grid that scores highest *on that benchmark's test set*, which is the strongest version of the baseline and the one least favourable to us, but it is still a choice informed by the test data, and no held-out split was used to make it. Second, the two methods are chosen differently: EWC-LoRA is re-tuned per benchmark, whereas FOLoRA's single (λ, k) is fixed on CIFAR-100 and then carried to ImageNet-R unchanged. The asymmetry works against us on ImageNet-R and in favour of the baseline there, and we state it so that the transfer claim in the abstract is read as what it is — one setting applied without retuning, not a setting validated on both benchmarks before the fact. Derivative-free and metaheuristic optimizers are the usual al-

ternative when the search space is too large to enumerate [76, 77, 78, 79, 80]; we do not use one, because a search whose trajectory cannot be enumerated would make the baseline tuning above harder to audit rather than easier.

Metrics and evaluation protocol. We evaluate every method under a common frozen-feature nearest-class-mean (NCM) protocol: after the final task we L2-normalize the CLS feature of the adapted backbone and classify by the nearest class mean computed from the training features of each task seen so far, with each task's means computed from that task's own training split. The protocol is standard in this literature [6, 7, 19], but it is *not* the one under which those methods are reported in their original papers — L2P, CODA-Prompt and InfLoRA each report a classifier learned jointly with their prompts — so every prompt-based and adapter-based row in Table 1 is a *re-evaluation* of that method by us under one decision rule, not a reproduction of its published number. We adopt the common rule because it removes the classifier head from the measurement: at the end of a twenty-task sequence the jointly trained head retains only about an eighth of what the frozen features support — 7.2–9.2% against 1% chance on CIFAR-100, which is 10–13% of the same model's NCM accuracy (Table 2) — so a head-based comparison would largely measure head drift rather than representation quality. Two consequences should be read alongside the numbers. First, the class means are recomputed at evaluation time from each task's training split, so the protocol assumes *evaluation-time* access to past training data. No gradient is ever computed on a previous task — this is not training-time replay — but it is an assumption, and we return to it in Sec. 5.6. Because the assumption is load-bearing, we also measured what it buys. In a strictly sequential variant

the class means are computed and frozen at the end of the task that produced them, so no past training set is read at evaluation; on the same three seeds the strict protocol costs 8.5 points for FOLoRA (69.43 ± 6.83 against 77.96 for the same runs under the standard rule), 14.0 for EWC-LoRA (64.40 ± 5.69 against 78.42) and 26.8 for Seq-LoRA (42.41 ± 6.17 against 69.17). The two protocols rank the methods the same way, and the gap is largest where the representation drifts most; Sec. 5.6 states this in full rather than leaving the standard-rule numbers to be read as replay-free. Second, because the decision rule is applied uniformly, the rows are comparable to each other but not to the published literature; the learned-head figures are given in Table 2 for reference. We report average accuracy over all tasks after the final task ($\overline{\text{ACC}}$) and the CIL–TIL gap (GAP). GAP is the mean drop, in points, when a task's test set is classified against all seen classes rather than against that task's own classes, under the same final-model feature extraction: it measures how far a method's decision rule is from relying on task identity, and lower is better. Because every method is run on the same seed set—and therefore sees the same class order and initialization—we assess the significance of differences between methods with a paired t-test over the seeds common to both. Seed budgets are *not* equal across methods and we do not pool them: EWC-LoRA and FOLoRA have $n = 10$ on both benchmarks; Seq-LoRA and O-LoRA $n = 10$ on CIFAR-100 and $n = 5$ on ImageNet-R; InfLoRA 5/3; L2P 5/5; CODA-Prompt 5/3. SimpleCIL is a deterministic floor ($n = 1$, no variance) and is not tested. Every p-value below states the common-seed count it was computed on, and Table 1 repeats the budget for each method.

Two exemplar-free methods are adjacent enough to warrant a word. FeCAM [81] and EFC [82] are among the closest exemplar-free continual learners to the setting studied here, and neither is a baseline of Table 1 — for reasons that are properties of our protocol rather than judgements of their quality. FeCAM contributes a *classifier*: it fits anisotropic class prototypes to frozen features, which is exactly the regime our NCM head occupies, so adding it would replace the evaluation rule that every row currently shares and would therefore measure a classifier rather than the training-time regularizer this paper varies. EFC contributes a mechanism that requires the backbone *not* to be frozen — its feature-drift regularizer exists to hold a moving backbone's features stable — so running it under frozen features would delete the mechanism under test. Holding the classifier and the backbone fixed across every method is what makes the comparisons in Table 1 isolate the training procedure; we accordingly claim nothing against FeCAM or EFC as complete systems, and comparing against either would call for a different experimental design than the one used here. Significance is reported on Holm–Bonferroni corrected p-values, with the uncorrected value given alongside whenever a claim turns on the difference. Table 1 fixes the confirmatory family, because it asks one question per baseline: within a benchmark and a metric, the contrasts against the nine CIFAR-100 and eight ImageNet-R baselines are one family, adjusted step-down over the sorted p-values. We do not pool the two benchmarks into a single family of seventeen tests. They answer different questions—on CIFAR-100 the comparison is to a baseline at parity, on ImageNet-R it is not—and pooling would inflate the family to buy protection against an error mode that neither table introduces. Elsewhere—

the EWC λ grid of Table 3, the ablations of Sec. 5.5, the O-LoRA variants of Sec. 5.6—the comparison is exploratory and the family is stated where the number is used. The correction removes no claim the argument rests on. The verdict pre-registered before any seed was run, ImageNet-R against EWC-LoRA at $\lambda = 100$, survives at $p_{\text{Holm}} = 0.0028$ on the CIL–TIL gap and 0.0011 on average accuracy; its CIFAR-100 counterpart was never claimed as significant and remains so ($p = 0.0734$, corrected 0.2934); and the paper's largest effect, the λ=0 ablation, survives a family more than three times larger than the confirmatory one ($p = 0.0005$, corrected 0.011 across all 28 CIFAR-100 configurations). What the correction does remove are marginal forgetting differences against L2P, CODA-Prompt and the weaker EWC-LoRA operating points ($0.023 \leq p \leq 0.040$); none is claimed below, and each is named where it arises. The correction now admits one forgetting contrast it previously did not, against O-LoRA, which runs in FOLoRA's favour once every per-task adapter is evaluated.

5.2. Main results

The protocol-independent facts hold across both tables: all methods share one frozen backbone and one classifier head. What separates the families is the per-task budget: FOLoRA, Seq-LoRA, EWC-LoRA, L2P and CODA-Prompt reuse one fixed set of trainable parameters across tasks, whereas O-LoRA and InfLoRA allocate a new adapter per task, so their *allocated* state grows linearly with the number of tasks (Table 7, frozen and cumulative columns). The distinction matters because the two are often conflated: O-LoRA's growth is entirely in frozen state — at T=20 it trains exactly the same 814,180 parameters as FOLoRA — and InfLoRA trains fewer still (224,356),

Table 1: Class-incremental results under the frozen-feature nearest-class-mean protocol (mean ± std over seeds; $\overline{\text{ACC}}$ higher is better). The bracketed interval under each entry is the 95% confidence interval of that mean ($t_{0.975,n-1}\,\text{sd}/\sqrt{n}$, n as below); SimpleCIL has n=1 and therefore no interval. GAP is the CIL–TIL gap: the mean drop, in points, when a task's test set is classified against all seen classes rather than its own, under a single final-model feature extraction (lower is better: less reliance on task identity). EWC-LoRA is shown at the best of its tuned points on each benchmark — λ=300 (CIFAR-100); λ=1000 (ImageNet-R) — i.e. the strongest configuration of the baseline; its full λ curve, including the pre-registered judging baseline λ=100, is in Table 3. The accuracy and gap comparisons at every tuned point are reported in Sec. 5.2. Seed budgets are *not* equal: EWC-LoRA and FOLoRA n=10 on both benchmarks; Seq-LoRA and O-LoRA 10/5 (CIFAR-100/ImageNet-R); InfLoRA 5/3; L2P 5/5; CODA-Prompt 5/3; SimpleCIL is the deterministic floor (n=1, no variance, untested). Bold marks the best value *among the rows shown*; ties are not bolded.

	Split CIFAR-100		Split ImageNet-R	
Method	$\overline{\text{ACC}}$	GAP	$\overline{\text{ACC}}$	GAP
Seq-LoRA	69.31±1.92 [67.9, 70.7]	9.92±0.58 [9.5, 10.3]	56.58±1.49 [54.7, 58.4]	8.83±0.70 [8.0, 9.7]
EWC-LoRA	**78.00±0.54** [77.6, 78.4]	**7.35±0.37** [7.1, 7.6]	66.33±0.61 [65.9, 66.8]	**7.00±0.30** [6.8, 7.2]
O-LoRA	64.94±3.03 [62.8, 67.1]	11.03±0.82 [10.4, 11.6]	53.72±0.65 [52.9, 54.5]	9.72±0.35 [9.3, 10.2]
L2P	73.65±0.81 [72.6, 74.7]	8.12±0.37 [7.7, 8.6]	59.47±0.43 [58.9, 60.0]	8.02±0.36 [7.6, 8.5]
CODA-Prompt	74.71±0.71 [73.8, 75.6]	7.71±0.55 [7.0, 8.4]	59.31±0.22 [58.8, 59.9]	7.83±0.42 [6.8, 8.9]
InfLoRA	72.95±0.64 [72.2, 73.7]	9.60±0.44 [9.0, 10.1]	58.93±0.19 [58.5, 59.4]	8.86±0.52 [7.6, 10.1]
SimpleCIL (floor)	70.31±0.00	8.48±0.00	49.71±0.00	7.96±0.00
FOLoRA (ours)	77.87±0.90 [77.2, 78.5]	7.42±0.59 [7.0, 7.8]	**66.37±0.77** [65.8, 66.9]	7.27±0.49 [6.9, 7.6]

Table 2: The learned classifier head after the final task, on the same trained models as Table 1. *Head* is the accuracy of the jointly trained linear classifier over all classes seen (1% chance on Split CIFAR-100, 0.5% on Split ImageNet-R); *NCM* is the frozen-feature nearest-class-mean accuracy of the same adapted backbone. Every method retains only about an eighth of its NCM accuracy in its own head (10–13% across both benchmarks), which is the reason we adopt a common NCM decision rule for the comparison rather than each method's published classifier. Means over the seed budgets listed in Table 1; the floor has no trained head.

	Split CIFAR-100		Split ImageNet-R	
Method	Head	NCM	Head	NCM
Seq-LoRA	7.19	69.31	6.55	56.58
EWC-LoRA	8.82	78.00	7.41	66.33
O-LoRA	7.47	64.94	6.64	53.72
L2P	8.74	73.65	6.98	59.47
CODA-Prompt	8.66	74.71	7.02	59.31
InfLoRA	9.23	72.95	6.17	58.93
FOLoRA (ours)	8.87	77.87	7.15	66.37

Table 3: EWC-LoRA regularization sensitivity. Each λ is compared against FOLoRA with a paired t-test over the seeds common to both; n is that common count, *not* either side's own budget, and it is why the FOLoRA column repeats the same value on every row of a benchmark. λ=100 is the baseline fixed in advance, before the seed top-up that closed the grid to n=10; λ=3000 was not run on ImageNet-R. The printed p is uncorrected; * marks $p < 0.05$ *after* Holm–Bonferroni correction within the confirmatory family of Table 1 (Sec. 5.1), so a row may print $p < 0.05$ without a star. The accuracy comparison is parity at every point tested *except* ImageNet-R at λ=100 (+2.85, $p_{\text{Holm}} = 0.0011$), where the paired difference reaches significance after correction. On CIFAR-100 the difference changes sign across the grid (+0.65 at λ=100 to −0.14 at λ=300), so the ordering of the two methods is not stable across the baseline's own tuned points. On the CIL–TIL gap FOLoRA has the *smaller* gap at the 1 tuned point where the difference survives correction, and EWC-LoRA at none: ImageNet-R λ=100 (FOLoRA 7.27 against EWC-LoRA 8.44, $p_{\text{Holm}} = 0.0028$).

			$\overline{\text{ACC}}$			
Benchmark	EWC λ	n	EWC	FOLoRA	Δ	p
CIFAR-100	100	10	77.22±1.08	77.87±0.90	+0.65	0.0734
CIFAR-100	300	10	78.00±0.54	77.87±0.90	-0.14	0.6440
CIFAR-100	1000	10	77.65±0.42	77.87±0.90	+0.21	0.4961
CIFAR-100	3000	10	77.40±0.62	77.87±0.90	+0.47	0.2507
ImageNet-R	100	10	63.52±1.28	66.37±0.77	+2.85	0.0002*
ImageNet-R	300	10	66.32±0.62	66.37±0.77	+0.05	0.9010
ImageNet-R	1000	10	66.33±0.61	66.37±0.77	+0.04	0.9235
ImageNet-R	3000	—	—	66.37±0.77	—	—

because it designs the down-projection A instead of learning it. The per-task advantage FOLoRA claims over these two is therefore one of *footprint*, not of trainable parameter count. The classifier head grows with the number of classes seen for *every* method, because the class-incremental protocol requires it and all methods share it; it is trained but discarded at evaluation under the NCM protocol, which is why the trainable column rises between T=1 and T=20 for every row. Fig. C.1 and Fig. C.2 (Appendix C) take the same final representations and re-score them as the candidate class set grows; we introduce them together with the semantics of that axis, because it is easy to misread as a training trajectory. Each figure shows the candidate-set restriction curves that the corresponding column of Table 1 collapses into a single final-task average. Both figures re-score a *single* feature extraction of the terminal model: the horizontal axis is the number of candidate classes the nearest-class-mean decision must choose among (5 per task on CIFAR-100, 10 on ImageNet-R, up to 100 and 200 respectively), *not* elapsed training and *not* the state of the model at that point of the sequence. The leftmost point therefore already contains the forgetting accumulated over the whole sequence, and the curve shows what happens to one fixed representation as the candidate set grows. On CIFAR-100 all six curves lie within five points of each other at the smallest candidate set (93.6 to 98.2) and span 64.9 to 78.0 at the full one; FOLoRA and EWC-LoRA in particular are indistinguishable at every candidate set, their paired difference ranging over only 0.66 points and never exceeding 0.46 in either direction. On ImageNet-R the spread at the smallest candidate set is already 7.8 points (82.9 for CODA-Prompt to 90.7 for EWC-LoRA, with FOLoRA at 90.5 and O-LoRA at 83.2), and

FOLoRA's difference from EWC-LoRA stays small as the candidate set grows rather than widening: -0.19 at the smallest set, -1.12 at the fifth task's set, $+0.04$ at the full set, a spread of 1.15 points — which is why we do not read that accuracy difference as an advantage that compounds with the number of tasks.

FOLoRA matches the strongest regularized baseline; it does not beat it. Under the comparison fixed in advance — a paired t-test against EWC-LoRA at $\lambda = 100$ — FOLoRA is ahead by $+0.65$ points on CIFAR-100 ($p = 0.0734$, which the pre-registered rule calls "matches or slightly exceeds" and *not* "outperforms") and by $+2.85$ points on ImageNet-R ($p = 0.0002$, all ten per-seed differences positive), which the same rule calls a significant lead. We report that verdict, and also why it must not be read as a general advantage: λ is a hyperparameter of the *baseline*, and the comparison moves with it. On CIFAR-100 EWC-LoRA's four tuned points span 77.22 to 78.00 while FOLoRA scores 77.87 — inside that span — with paired differences of -0.14 ($p = 0.644$) at $\lambda = 300$, $+0.21$ ($p = 0.496$) at $\lambda = 1000$ and $+0.47$ ($p = 0.251$) at $\lambda = 3000$. That is parity at every CIFAR-100 point, and EWC-LoRA's best point sits 0.13 points *above* FOLoRA. On ImageNet-R the same structure appears with a wider spread, and the lead is confined to one end of it: $\lambda = 100$ is that benchmark's weakest EWC point at 63.52, and it is there — and only there — that the comparison reaches significance, while at $\lambda = 300$ and $\lambda = 1000$, both roughly 2.8 points higher, the difference collapses to $+0.05$ ($p = 0.901$) and $+0.04$ ($p = 0.924$). The sign of our difference against EWC-LoRA is therefore set by which operating point of the baseline is read, so we report the whole curve (Table 3) rather than

the point that reads best for us. EWC-LoRA was not run at $\lambda = 3000$ on ImageNet-R.

On the CIL–TIL gap the comparison is mixed, and we report both directions. The direction is set by the baseline's regularization strength, not by the method. On CIFAR-100 EWC-LoRA's gap is the smaller one at the two larger strengths, but neither difference survives the correction of Sec. 5.1: the paired gap is +0.32 points at $\lambda = 1000$ and +0.39 at $\lambda = 3000$, nominally significant at $p = 0.0400$ and 0.0401 and corrected to 0.16 in both cases, so on this benchmark the two methods are indistinguishable in the gap direction. At $\lambda = 300$ the difference is +0.07 ($p = 0.568$) and at the pre-registered $\lambda = 100$ it is -0.30 ($p = 0.071$), where the sign runs the other way without reaching significance either way. On ImageNet-R the direction reverses and there the difference survives: FOLoRA has the significantly *smaller* gap at $\lambda = 100$ (-1.17 points, $p = 0.0004$, corrected 0.0028). At $\lambda = 300$ the nominal advantage (-0.41, $p = 0.0317$) does not survive correction (0.13), and at $\lambda = 1000$ the two are indistinguishable (+0.27, $p = 0.203$). Because average accuracy and the gap are measured under the same final feature space—a representation that separates the classes better both classifies better and loses less when the candidate set grows—the two co-vary across these methods. We therefore do not treat the gap as an independent axis of merit in either direction; we report it rather than leaving a reader to assume the favourable direction.

FOLoRA is ahead of the parameter-efficient family, and there the difference is uniform. Against L2P, CODA-Prompt, InfLoRA and O-LoRA the paired differences in average accuracy are significant on both

benchmarks with every per-seed difference of the same sign: across the four baselines the mean paired difference runs from +3.61 to +12.92 points on CIFAR-100 (corrected $p \leq 0.008$) and from +7.45 to +13.20 on ImageNet-R (corrected $p \leq 0.007$). The upper end of each range is set by the initialization-only O-LoRA row, and it narrows against the training-time orthogonalized objective that the original paper specifies, which we also ran: on CIFAR-100 to +7.10 and +5.60 at the two constraint strengths (λ_1=0.1 and λ_1=1.0; corrected $p < 0.001$ for both within the two-member family those variants form), and on ImageNet-R to +10.59 and +5.74 (corrected $p = 0.034$ each in the same family). This is the paper's positive empirical result. It is also the comparison in which the baselines' seed budgets are smallest ($n = 5$, or $n = 3$ for CODA-Prompt and InfLoRA on ImageNet-R), which is why Sec. 5.1 lists each budget separately instead of implying a shared one — and why the claim is stated against the parameter-efficient family rather than against full-parameter regularization. Under the NCM rule used throughout this table two rows sit below the frozen-feature floor of 70.31: O-LoRA at its initialization-only setting on CIFAR-100 (64.94) and Seq-LoRA (69.31, unprotected sequential training of the shared adapter). Only the first is decisive. O-LoRA's gap is 5.4 points and is significant against the constant floor ($p = 0.0003$, one-sample test over ten seeds); Seq-LoRA's is 1.0 point and is not ($p = 0.13$). We therefore do not rest an argument on Seq-LoRA's shortfall — it is a nominal sign, not evidence — and the claim that the forgetting this paper targets lives in the adapter rather than in the head is carried by the direct temporal measurement of Sec. 5.4, not by this comparison. Sec. 5.3 measures how much of the floor comparison depends on

the decision rule: under a stronger training-free head neither shortfall is detectable, and the ordering between the two strongest rows does not change. Every other row clears the floor: InfLoRA at 72.95 on CIFAR-100 and 58.93 on ImageNet-R, O-LoRA at 53.72 on ImageNet-R, against floors of 70.31 and 49.71 respectively; and the training-time orthogonality term lifts the O-LoRA mean above the CIFAR-100 floor at both constraint strengths (71.21 and 72.72); Sec. 5.6 reports those configurations in full.

5.3. How much of the ranking is the decision rule?

Every number in Table 1 is produced by one decision rule — cosine nearest-class-mean on terminal features — and it is fair to ask whether the ordering is a property of the representations or of the rule. Table 4 separates the two by re-scoring the *same* features with RanPAC [70], a training-free ridge classifier over M=10,000 random projections whose regularization is selected per task on a held-out split of that task's training features. Nothing is retrained and no adapter hyperparameter changes: the two columns of a row come from a single checkpoint, and the NCM column is recomputed from the same cached features rather than quoted from Table 1. Before a RanPAC number is accepted we verify that the cached features reproduce the published NCM accuracy of that exact run; all rows reproduce to the second decimal, which rules out the silent failure mode in which an adapter is not fully restored and the evaluation quietly falls back to the frozen backbone.

The decision rule dominates the absolute level on both benchmarks: every configuration gains between 5.3 and 13.7 points, and the frozen backbone alone reaches 81.57 on CIFAR-100 and 60.00 on ImageNet-R. On the same checkpoints, that is above FOLoRA's NCM figure of 77.83 on CIFAR-100

Table 4: Decision rule × representation. Each row re-evaluates the *same* features with two rules: cosine nearest-class-mean (NCM, the protocol used throughout the paper) and RanPAC (a training-free ridge classifier with random projections, NeurIPS 2023) fitted on the same features. No model is retrained for the RanPAC columns.

	CIFAR-100		ImageNet-R	
Feature extractor	NCM	RanPAC	NCM	RanPAC
Frozen ViT-B/16 (no adaptation)	70.31	$81.57_{\pm 0.47}$	49.71	$60.00_{\pm 0.15}$
Seq-LoRA	68.22	$81.95_{\pm 0.80}$	58.78	$66.73_{\pm 0.84}$
EWC-LoRA (tuned)	77.41	$85.04_{\pm 0.39}$	66.87	$72.15_{\pm 0.49}$
O-LoRA	67.78	$80.80_{\pm 0.90}$	54.61	$64.53_{\pm 0.77}$
InfLoRA	72.51	$82.54_{\pm 0.15}$	59.00	$64.96_{\pm 0.47}$
FOLoRA (ours)	77.83	$84.98_{\pm 0.16}$	66.43	$71.74_{\pm 0.44}$

Both columns use the same terminal checkpoint per row; the NCM column is seed 0, so it can differ from the multi-seed mean of the same configuration in Table 1 — by up to 2.8 points (O-LoRA, sd 3.03 over 10 seeds on CIFAR-100) — and a row should be read within this table rather than against Table 1. The RanPAC column is the mean±std over the three class orders (seeds 0–2) on both benchmarks. M=10,000 random projections, λ selected per task on a held-out split of the current task. EWC uses the λ that is best tuned for each benchmark (λ=300 on CIFAR-100, λ=1000 on ImageNet-R), matching Table 1. FOLoRA vs. the frozen backbone under the RanPAC rule isolates what the adapter buys once the decision rule is no longer the bottleneck.

but 6.4 points below its 66.43 on ImageNet-R, so a stronger head does not by itself overtake adaptation on the harder benchmark. The gain is smallest for the two strongest adapters (5.3 and 7.2 points for FOLoRA, 5.3 and 7.6 for the tuned EWC-LoRA) and largest for the weakest (9.9 and 13.0 for O-LoRA, and 13.7 for Seq-LoRA on CIFAR-100): the stronger head compresses the differences among the methods that already sit at the top rather than creating them.

What survives the swap is the identity of the top two rows and FOLoRA's standing within the parameter-efficient family. Measured over three class orders on both benchmarks, FOLoRA and the tuned EWC-LoRA are indistinguishable under RanPAC on both: 84.98 against 85.04 on CIFAR-100 (paired difference -0.06 ± 0.32, $p = 0.78$) and 71.74 against 72.15 on ImageNet-R (-0.41 ± 0.79, $p = 0.46$). This is the same verdict the cosine rule gives on the ten-seed means (-0.14, $p = 0.64$ on CIFAR-100; $+0.04$, $p = 0.92$ on ImageNet-R), so the parity between the two strongest methods is a property of the representations and not of the decision rule. Against the rest of the parameter-efficient family FOLoRA is ahead under RanPAC on both benchmarks, by +2.44 over InfLoRA on CIFAR-100 and by +5.01 over Seq-LoRA on ImageNet-R, both at the narrow end and both at $p < 0.01$. What the adapter buys is therefore not an artefact of the cosine rule: FOLoRA plus RanPAC is 3.41 points above the frozen backbone on CIFAR-100 and 11.74 on ImageNet-R.

One secondary comparison is weaker than we first wrote it, and we give the corrected form rather than the version we would prefer. Under NCM, O-LoRA at its initialization-only setting is 5.4 points below the frozen-feature

floor and Seq-LoRA 1.0 point below it, of which only the first is significant; reading the second as "sequential adaptation ends below simply classifying frozen features" does not survive a test against the floor ($p = 0.13$, ten seeds). Under RanPAC the picture dissolves in both directions: the frozen backbone reaches 81.57, the three remaining adapted backbones lie within one point of it (Seq-LoRA 81.95, O-LoRA 80.80, InfLoRA 82.54) against a run-to-run standard deviation of 0.89 points for the difference of two runs, and only the two strongest adapted rows clear it by more than that band (EWC-LoRA 85.04 and FOLoRA 84.98, +3.5 and +3.4). The floor therefore separates nothing in the middle of the table once the rule changes. ImageNet-R has no such ambiguity to resolve: under NCM every adapted backbone already clears the floor (49.71) by at least 4.0 points (ten-seed means, Table 1), and under RanPAC by at least 4.5 (64.53 to 72.15 against 60.00), so the floor discussion is a CIFAR-100 question on either rule. What carries the paper is the direct temporal measurement of Sec. 5.4: Seq-LoRA falls 21.71 points below its own per-task accuracy and FOLoRA 10.39, and that gap does not depend on which classifier reads the features.

We also do not claim that Table 1 reports the best achievable number on these benchmarks. Read with the same head, the frozen backbone reaches 81.57 on CIFAR-100, which only the two strongest adapted rows clearly beat (85.04 and 84.98; InfLoRA's 82.54 sits at the edge of the noise band and Seq-LoRA's 81.95 inside it), and on ImageNet-R all five adapted rows beat its 60.00. So adaptation is not redundant, and a practitioner whose only objective is accuracy on CIFAR-100 may still prefer the frozen backbone with a trained-nowhere classifier to most of the PEFT-CL rows. What Table 1

measures is what adaptation buys under a common, parameter-free decision rule, which is the question this paper is about.

5.4. True temporal forgetting

The second column of Table 1 is a CIL–TIL gap, not a forgetting measure. As set out in Sec. 5.1 it is computed from a single feature extraction of the *terminal* model re-scored against candidate sets of increasing size, so it cannot separate "the model forgot task j" from "the model was never asked to discriminate task j's classes against the later ones". To measure forgetting we re-ran four configurations with a per-boundary protocol: after each of the 20 tasks, the features of every test image are extracted from the model *as it stands at that boundary* and scored by NCM over the classes seen so far. This yields a full $T \times T$ matrix A whose entry $A_{t,j}$ is the accuracy on task j's test set after learning task t, and from it

$$\mathrm{BWT} = \frac{1}{T-1} \sum_{j<T-1} \big(A_{T-1,j} - A_{j,j}\big), \qquad \mathrm{PD} = \frac{1}{T} \sum_{j} \Big(\max_{t \geq j} A_{t,j} - A_{T-1,j}\Big), \tag{19}$$

both in points, negative BWT meaning forgetting. BWT is the standard definition; the peak-drop measure PD adds a penalty when a task's accuracy falls after it was learned and later recovers, which BWT on its own can hide. Table 5 reports both, together with the average incremental accuracy (AIA, the mean over boundaries of the accuracy the model then had on every class seen so far) and the final accuracy of the same runs, so that forgetting is not read in isolation from what the method achieves nor from the trajectory that produced it.

Four findings. First, the method roughly halves the forgetting of unpro-

Table 5: True temporal forgetting on Split CIFAR-100. Unlike the CIL–TIL gap in Table 1, every entry here comes from a feature extraction at each of the 20 task boundaries, so $A_{t,j}$ is the accuracy measured with the model as it stood *then*. BWT $= \frac{1}{T-1}\sum_{j<T-1}(A_{T-1,j} - A_{j,j})$; the peak-drop measure adds a penalty when a task's accuracy falls after it was learned and later recovers. Negative BWT means forgetting; both are in percentage points, mean±sd over three seeds. AIA is the average incremental accuracy, the mean over boundaries of the accuracy the model then had on every class seen so far; final accuracy is from the same runs, so the last three columns are directly comparable within a row but are not the 10-seed means of Table 1; λ=300 is the tuned EWC-LoRA row of that table.

Configuration	BWT ↓	Peak drop ↓	AIA ↑	Final acc.	n
EWC-LoRA (λ=1000)	-7.73 ± 0.72	$+7.84 \pm 0.70$	85.77 ± 0.96	78.05 ± 0.45	3
EWC-LoRA (λ=300, tuned)	-8.75 ± 0.60	$+8.82 \pm 0.55$	85.22 ± 1.32	77.91 ± 0.55	3
FOLoRA (ours)	-10.39 ± 1.15	$+10.40 \pm 1.16$	84.08 ± 1.25	77.28 ± 0.77	3
Seq-LoRA	-21.71 ± 1.89	$+21.71 \pm 1.89$	80.01 ± 1.27	68.54 ± 1.97	3

tected sequential training: BWT improves by 11.33 points ($p = 0.003$ paired over three seeds) and FOLoRA also ends 8.74 points higher on final accuracy ($p = 0.012$). Second, FOLoRA is *not* the best-forgetting method in the table. Against the tuned EWC-LoRA of Table 1 (λ=300) neither its BWT (-1.64, $p = 0.18$) nor its final accuracy (-0.63, $p = 0.47$) differs significantly, and against the more strongly regularized λ=1000 variant it forgets significantly more (-2.65, $p = 0.013$, uncorrected, $n = 3$). We read this as the parity established in Sec. 5.2 extending to forgetting, not as evidence that FOLoRA protects memory better than full-parameter regularization. Third, the peak-drop measure tracks |BWT| to within 0.11 points in every row, so none of the four curves drops after a task is learned and later recovers: the ordering above is not an artefact of a single definition. Fourth, the trajectory measure agrees with the endpoint. AIA averages over all 20 boundaries rather than the last one, so it weights the mid-sequence points that neither BWT nor final accuracy sees: FOLoRA reaches 84.08 ± 1.25 against 85.22 ± 1.32 for the tuned EWC-LoRA (-1.14, $p = 0.28$) and 4.07 points above Seq-LoRA ($p = 0.013$), which is the same ordering as the other two columns.

5.5. Ablation study

We ablate FOLoRA's two hyperparameters — the regularization strength λ and the number of protected directions k — on Split CIFAR-100. Table 6 reports average accuracy and the CIL–TIL gap for each setting together with a paired t-test against the pre-registered main configuration (k=64, λ=3).

The penalty carries the entire effect. Setting $\lambda = 0$ removes the Fisher-weighted orthogonality penalty and changes nothing else; on the five seeds common to both configurations average accuracy falls from 78.32 to

Table 6: Ablation on Split CIFAR-100. Δ and p compare each setting with the pre-registered main configuration (k=64, λ=3) using a paired t-test over common seeds; n is each row's own seed budget, so Δ is computed on the intersection and need not equal the difference of the two means shown; it is taken as main configuration minus setting, so a positive Δ means the main configuration scores the higher of the two. λ=0 removes the Fisher-weighted orthogonality penalty and changes nothing else. GAP is the CIL–TIL gap of Table 1, defined there; lower is better. The printed p is uncorrected; * marks $p < 0.05$ *after* Holm–Bonferroni correction over all CIFAR-100 configurations tested (Sec. 5.1), so a row may print $p < 0.05$ without a star.

Setting	n	$\overline{\text{ACC}}$	GAP	Δ vs main	p
λ *at* k=64					
λ=0	5	69.59±1.55	9.53±0.65	+8.72	0.0005*
λ=1	5	77.33±1.13	7.56±0.80	+0.99	0.1106
λ=3	10	77.87±0.90	7.42±0.59	*(main)*	—
λ=10	5	78.24±0.64	6.87±0.30	+0.08	0.8868
k *at* λ=3					
k=16	5	78.14±0.68	7.32±0.61	+0.17	0.7487
k=64	10	77.87±0.90	7.42±0.59	*(main)*	—
k=128	5	77.66±0.63	7.49±0.42	+0.65	0.2768

69.59 ($\Delta = +8.72$, $p = 0.0005$), i.e. to the level of unprotected sequential fine-tuning (69.31, Table 1). (The λ=3 full ten-seed mean is the 77.87 of Table 1; the two figures differ only in seed budget, not in configuration.) The benefit of FOLoRA is therefore attributable to the regularization term rather than to the shared adapter, the rank constraint or the training recipe — which is what the derivation predicts: with no penalty the protected subspace is empty and the method degenerates to Seq-LoRA.

The direction of the subspace matters, though three seeds do not establish by how much. A penalty on *some* fixed subspace would also restrain the adapter, so we ran the main configuration with the protected subspace replaced by a random one — same per-layer rank k=64, same strength λ=3, same scale, nothing else changed. Protection along random directions reaches 74.35 ± 1.43 against 78.37 ± 0.84 for the Fisher subspace on the three common seeds, a paired difference of -4.02 points ($p = 0.070$, n=3). The sign is the one the method predicts — a subspace chosen by curvature is worth roughly four points more than one chosen at random — but with three seeds and a paired spread of 1.95 points it does not reach significance, and we report it as a direction rather than as a demonstration. What the comparison does rule out is the reading that the penalty works merely by shrinking the adapter: an equally strong penalty on an uninformative subspace recovers only part of the effect.

At k=64 the λ response is a step, not a peak, over the range we probed. Accuracy rises from 69.59 at $\lambda = 0$ to 77.33 at $\lambda = 1$ and 77.87 at $\lambda = 3$, and does not move beyond that: $\lambda = 10$ prints 78.24 on the five seeds it was run on, but on those same five seeds the main configuration is ahead

by +0.08 ($p = 0.887$) in the paired test — the two are indistinguishable, and the 0.37-point gap between the printed means is a seed-budget difference, not a gain. Only $\lambda = 0$ separates from the main configuration ($p = 0.0005$); $\lambda \in \{1, 10\}$ are each within noise of it ($p = 0.111$ and $p = 0.887$). We therefore do not claim an interior optimum at k=64 — the grid does not resolve one, and any ordering among $\lambda \in \{1, 3, 10\}$ is inside the run-to-run band of Sec. 5.6. The paired tests behind these accuracies are in Table 6.

At k=16 the decline at large λ is visible. Extending the sweep on the smaller protected subspace, where the penalty acts on fewer directions, average accuracy peaks at $\lambda \in \{3, 10\}$ (78.14 and 77.90, both n=5) and then declines before plateauing: 75.35 at $\lambda = 100$ (n=3), 73.61 at $\lambda = 300$ (n=10) and 73.85 at $\lambda = 1000$ (n=3) — the last two sit inside the run-to-run band, so we read the response as saturating rather than continuing to fall. This is the interior optimum that Remark 2 predicts, and it is the reason we treat the large-λ end as a real failure mode rather than as a harmless over-regularization. The seed budgets differ across these points, so we read the shape rather than the individual gaps; the paired tests behind them are in Table A.10.

k is a memory–accuracy dial, not a knob to tune. Accuracies at $k \in \{16, 64, 128\}$ differ by less than one point (78.14, 77.87, 77.66) and no paired difference is significant ($p \geq 0.2768$, the k=128 comparison; Table 6). The same insensitivity holds on the harder benchmark: 67.46 ± 0.35, 66.37 ± 0.77 and 67.47 ± 0.12 at $k = 16$, 64 and 128 ($n = 3$, 10 and 3). Over the three common seeds the paired differences from the main configuration are -0.57 ($p = 0.311$) and -0.58 ($p = 0.199$) in the convention of Table 6

(main configuration minus the setting), i.e. the two other ranks are marginally ahead — again inside the noise, and in the opposite direction to CIFAR-100, as a difference of this size would be. The importance state grows linearly in k — 47.8 MiB at k=16, 182.8 MiB at k=64 and 362.8 MiB at k=128 on the accounting convention of Table 7 (protected directions plus reference snapshot). We report k=64 because it was the pre-registered main configuration, not because it scores highest — it does not. Since k buys no measurable accuracy, the practical recommendation for a memory-constrained device is k=16 at $\lambda \in \{3, 10\}$; rank selection under a deployment budget has been studied for quantized adapters [83], and here the memory is spent on the importance state rather than on the weights.

Fisher weighting moves the strength at which the plateau is reached, not the plateau. Setting $\sigma_j \equiv 1$ and changing nothing else gives a second arm on the same λ grid — ten seeds at every strength for the equal-weight arm, and for the weighted arm ten at $\lambda = 3$ and five at the others. At small strengths the weighted arm is clearly ahead: +3.83 points at $\lambda = 1$ ($p = 0.012$, five paired seeds) and +4.19 at $\lambda = 3$ ($p = 6 \cdot 10^{-7}$, ten). The gap closes at $\lambda = 10$, where the paired difference is +0.38 ($p = 0.687$, five seeds); over all ten seeds the equal-weight arm sits at 76.66 ± 2.37 there, against the weighted arm's 78.24 ± 0.64 over its five (77.86 for the equal-weight arm over the same five), so what separates the two at this strength is spread rather than level. The fourth strength divides them rather than confirming them: at $\lambda = 30$ the weighted arm falls to 75.02 while the equal-weight arm reaches 77.93 over its full ten seeds (77.81 over the five the two share) — the paired difference is -2.79 ($p = 0.004$). The weighting therefore buys an earlier and

narrower optimum, not a higher one. The weighted arm is on its plateau by $\lambda = 3$ and has left it by $\lambda = 30$, whereas the equal-weight arm climbs monotonically from 73.11 at $\lambda = 1$ to 77.93 at $\lambda = 30$, arriving at the same plateau somewhere in $\lambda \in [10, 30]$ — a factor of three to ten further along the axis. Inside the weighted arm's window the spread is also tighter (sd 0.64–1.13 over $\lambda \in \{1, 3, 10\}$ against 1.43–2.37 for the equal-weight arm at the same strengths). At their own best strengths on this grid — $\lambda = 10$ for the weighted arm and $\lambda = 30$ for the equal-weight arm — the two are indistinguishable ($+0.43$, $p = 0.44$, five paired seeds). We do not claim — and Sec. 4.6 does not claim — that the weighting raises the achievable accuracy over a *well-tuned* equal-weight penalty, because the weighted family contains the equal-weight geometry as the case $\sigma_j \equiv 1$. The difference our data support lies in the operating point rather than in the ceiling: one λ reaches the plateau for the weighted formulation, which is the property the same-hyperparameter protocol of Sec. 5.2 uses across benchmarks, whereas the equal-weight arm is far below it at $\lambda \leq 3$ — and at the far end the trade reverses, since the weighted formulation is the one that requires λ to stay inside its window. The matched-λ pairings are listed in Table A.10, at the common-seed count rather than at either arm's full budget. The same contrast at the smaller protected rank is much flatter. Over the three seeds the two arms share, the weighted arm is 77.94 against 77.17 at $\lambda = 3$ ($+0.76$, $p = 0.24$) and 77.63 against 77.99 at $\lambda = 10$ (-0.36, $p = 0.0099$); over its full five seeds the weighted arm is 78.14 ± 0.68 and 77.90 ± 0.71 at the two strengths. The ordering changes sign between $\lambda = 3$ and $\lambda = 10$, and the $\lambda = 10$ difference — statistically detectable before correction, but less than

half the 0.89-point resolution of the protocol — is the same conclusion the k=64 grid reaches, and the reason we do not read either sign as an accuracy claim.

5.6. Limitations and discussion

Reproducibility floor. We measured the run-to-run spread directly rather than inferring it from a single pair of runs. On CIFAR-100, at seed 0 and with a fixed configuration, we ran the EWC-LoRA configuration seven times and the FOLoRA configuration four times; the `config.json` files of each group agree field by field except for the `tag`, and the one other field that varies (`folora_lambda`) is not read by the EWC method at all, which we confirmed in the source. The single-run standard deviation is 0.69 points for EWC-LoRA (range 75.97–78.06 over seven runs) and 0.47 for FOLoRA (range 77.22–78.37). The two are of the same order (variance ratio F=2.16, p=0.56), so we pool them: s=0.63 over eleven runs. A difference between two independent runs therefore has a standard deviation of $\sqrt{2}\,s = 0.89$ points, and that is the band drawn in Fig. C.1. Two consequences follow. First, the 1.29-point gap we quoted in an earlier version of this section — two runs at the same seed that produced 77.26 and 75.97 — is 2.1 s, not the floor: it is one pair out of the sample, and the two most extreme of the seven runs lie further apart still (78.06 and 75.97, 3.3 s). With a single pair there is no way to tell a floor from a tail, which is why we replaced the pair with repeated runs. Second, and this governs how the table should be read, the differences that decide the ordering of the *leading* rows are far inside the band: FOLoRA against tuned EWC-LoRA differs by 0.13 points on CIFAR-100, and either of them against InfLoRA by at least 4.9 points. A single seed, or

any subset of seeds chosen after the fact, therefore cannot support a claim in either direction at that end of the table. This is why we report paired t-tests over seeds throughout and why we describe any difference inside the band as parity rather than as a win — including differences that happen to favor us.

Equivalence, not merely non-significance. A paired t-test that fails to reject is not evidence of equality, so for the one comparison whose conclusion is *parity* — FOLoRA against tuned EWC-LoRA — we run a two one-sided test (TOST) instead of relying on a non-significant difference. The margin is δ=1.0 points, set at the resolution of a single comparison: it is 1.1 times the 0.89-point standard deviation of the difference of two independent runs measured above, so a smaller margin would claim an equivalence that no number of seeds could establish. Equivalence is established at both tuned EWC-LoRA points on both benchmarks — on CIFAR-100 -0.14 against λ=300 (p_{TOST}=0.008) and $+0.21$ against λ=1000 (p=0.014); on ImageNet-R $+0.05$ (p=0.015) and $+0.04$ (p=0.017) — and it is *not* established at the pre-registered λ=100 on ImageNet-R ($+2.85$, $p \approx 1$), which is the same setting where the accuracy difference is significant after correction; there the text reports a difference rather than parity.

Where the variance comes from. On CIFAR-100 the accuracy of FOLoRA is consistent in mean but noisier across seeds than the regularized baselines it is compared against — ± 0.90 for FOLoRA against ± 0.54 for EWC-LoRA and ± 0.64 for InfLoRA (Table 1) — while being *less* noisy than unprotected sequential training (± 1.92 for Seq-LoRA and ± 3.03 for O-LoRA at its initialization-only setting), which is consistent with the penalty damping seed-to-seed drift rather than amplifying it. We attribute the residual

variance to two sources. First, the importance kernel is estimated from a finite per-sample budget (up to 300 gradients per task), and its top-k approximation is sensitive to the particular samples that land in the tails of the estimate. Second, under a frozen backbone, a fixed-rank (r=16) adapter and no replay, the class ordering exerts a strong effect on the final subspace, so individual seeds drift apart. We further found that the epoch budget interacts with the method family: adapter-based Fisher methods degrade under longer training because over-fitting drives the per-sample gradients toward zero and thereby degenerates the estimated Fisher, whereas prompt-based methods benefit from longer training of their prompt keys. This motivates the per-family epoch budgets in Sec. 5.1. A natural direction for future work is a stochastic (Monte-Carlo) Fisher estimate and class-order averaging to tighten the variance.

What we claim, in two layers. Because the CIFAR effect sizes sit inside the noise band above, we separate the claims into one layer the evidence supports unconditionally and one it supports only with an explicit caveat.

The first layer is about *what the method does*, and does not depend on beating anyone. (i) Removing the Fisher-weighted penalty and changing nothing else returns the method to the level of unprotected sequential fine-tuning, so the entire effect of FOLoRA is carried by the regularization term it adds rather than by the shared adapter or the training recipe (Sec. 5.5). (ii) The protected rank is not a sensitive knob: accuracies at $k \in \{16, 64, 128\}$ differ by less than one point and none of the differences is significant, while the importance state grows linearly in k — which makes k a memory–accuracy dial rather than a hyperparameter to tune. (iii) On CIFAR-100 FOLoRA

reaches accuracy *on par with the strongest regularized baseline* rather than above it; we state parity as parity. Against EWC-LoRA, the comparison flips sign depending on which of the tuned EWC operating points is taken as the baseline, so we report all of them in Table 3 and let the reader see the flip rather than selecting the point that reads best.

The second layer is where FOLoRA is *ahead*, and it holds against the parameter-efficient family rather than against full-parameter regularization: against L2P, CODA-Prompt, InfLoRA and O-LoRA, the paired differences are significant on both benchmarks and seed-wise all of one sign. That claim is the one we treat as the paper's positive result; it is also the one where the baseline's seed budget is smallest, so Sec. 5.1 records the seed sets explicitly for each family instead of implying a single shared one.

Cost: trainable parameters are traded for importance state. FOLoRA's efficiency claim is narrower than "memory-efficient", and we quantify the trade-off rather than leaving it implicit. The trainable adapter is the same one at every task — it is reused and re-projected — so the only growth in the trainable column is the classifier head, which adds the same 0.073M to every method in Table 7 (0.741M at T=1 against 0.814M at T=20 for FOLoRA), and the low-rank update merges back into the frozen backbone, so inference costs nothing extra. Against that, the importance model is a per-layer rank-k subspace rather than EWC's per-parameter diagonal, and it is larger. Table 7 states both sides with a single accounting convention: the importance column counts *everything a method must keep resident to compute its penalty*, i.e. the Fisher plus the parameter anchor for EWC, and the accumulated directions plus the reference snapshot for FOLoRA. On that convention FOLoRA at

k=64 is 32.5× EWC's importance state; at k=16 it is 8.5×. This is the price of representing direction correlations, which a diagonal preconditioner cannot do at any rank, and it is why we report k as a dial rather than claiming a memory advantage. The *time* cost is much smaller than the memory cost: estimating the importance kernel adds roughly 5% to the wall-clock cost of an epoch over EWC-LoRA (28.2 against 26.8 s/epoch, medians over all seeds of the runs in Table 1), so the eigenvalue decomposition and the 300-sample Fisher estimate are not the dominant term — the training pass is. Prompt-based methods are cheaper per epoch (16.7–18.8 s) but are trained for four times as many epochs, and cost more per task in total (333–375 s against 141 s for FOLoRA).

Table 7 accounts for what each method must keep resident; it does not by itself say what the training process actually allocates. We therefore measured peak device memory directly, each method in its own process, on the same twenty-task CIFAR-100 sequence and with one epoch per task — the peak is set by the model, the optimizer state and a single batch, so the epoch count does not enter into it — and with every method, FOLoRA included, at the adapter rank of Table 1 (r=16). The four baselines peak between 1,977 and 2,436 MiB. FOLoRA at its main configuration (k=64) peaks at 2,598 MiB allocated (4,660 MiB reserved), which is 162 MiB — 6.6% — above the largest baseline and well inside the 8,151 MiB of dedicated memory the card reports. The allocator reserves more than that on every method (2,140–2,908 MiB for the baselines, 4,660 MiB for FOLoRA): this is a Windows/WDDM device, where reservations can be backed by shared system memory and are therefore not bounded by the card, so the allocated figure is the one we compare across

methods. The whole of FOLoRA's excess is the protected subspace. At the first task, before any subspace exists, its peak is 2,410 MiB — the same figure sequential LoRA and EWC-LoRA reach at that task — and from the second task on it rises by 187 MiB, which is the 182.8 MiB of Table 7 plus the 2.8 MiB reference snapshot to within 1.2 MiB. Running the same method at k=16 and changing nothing else holds the same first-task peak and grows by 50 MiB instead (47.8 MiB of state plus the snapshot, again to within about one MiB), so the 137 MiB that separate the two configurations is the protected rank, not the Fisher estimate — the estimate is a per-task cost that does not accumulate. A device that cannot spare the k=64 state therefore loses no measurable accuracy by taking k=16 (Sec. 5.5) and recovers that 137 MiB.

Inference is where the accounting reverses. A merged adapter is the backbone, so FOLoRA adds no tokens and no forward pass; the prompt methods extend every input by the number of prompt tokens they select, 25 for L2P (prompt length 5, top-5 of a 20-slot pool) and 40 for CODA-Prompt (length 8, top-5 of 100 components) in the configurations behind Table 1, and each of those tokens passes through the frozen backbone at every inference. Table 7 does not tabulate this column; the counts are computed from each run's stored configuration.

Scope. Two limitations bound the generality of everything above. First, the evidence covers two benchmarks and a single backbone (ViT-B/16); we make no claim about whether the curvature-weighted subspace transfers to convolutional backbones or to domain-incremental shifts, where the class-incremental protocol we evaluate under does not directly apply. Second, the baselines are reproduced under one shared protocol and one shared backbone

Table 7: Resource accounting in millions of parameters, keeping *trainable* (gradient computed), *frozen* (allocated and resident but not trained) and *cumulative* (the two added) apart. Only O-LoRA and InfLoRA allocate a module per task, and their growth is entirely frozen: at T=20 O-LoRA trains the same parameter count as FOLoRA, and InfLoRA trains fewer because it designs rather than learns the down-projection A. All rows carry the classifier head, which grows with the class count for every method. "Importance" is the state needed to compute the penalty: the diagonal Fisher plus the anchor for EWC-LoRA (2.8 MiB for the Fisher alone), and for FOLoRA the protected directions (45.0 MiB at k=16, 180.0 MiB at k=64, 360.0 MiB at k=128) plus a shared 2.8 MiB reference snapshot, which is what the 47.8, 182.8 and 362.8 of the column add to. "s/epoch" is the median wall-clock cost of one epoch per task over all seeds of the run behind the corresponding row of Table 1, rounded half up to one decimal. The reference run measured for each row is `experiments/cifar100/<method>/<tag>/seed0`, with tags `default` (Seq-LoRA, EWC-LoRA, O-LoRA, InfLoRA), `v2f_l3_k64` (FOLoRA), `pilot20` (L2P) and `pool100_len8_ep20` (CODA-Prompt); each run's `config.json` holds its full resolved configuration.

Method	Trainable		Frozen	Cumulative	Importance	
	T=1	T=20	T=20	T=20	state	s/epoch
Seq-LoRA	0.741	0.814	0	0.814	—	25.7
EWC-LoRA	0.741	0.814	0	0.814	5.62 MiB	26.8
O-LoRA	0.741	0.814	14.008	14.823	—	36.9
InfLoRA	0.151	0.224	14.598	14.823	—	37.2
L2P	0.115	0.188	0	0.188	—	18.8
CODA-Prompt	0.701	0.774	0	0.774	—	16.7
FOLoRA (k=16)	0.741	0.814	0	0.814	47.8 MiB	28.2
FOLoRA (k=64)	0.741	0.814	0	0.814	182.8 MiB	28.2
FOLoRA (k=128)	0.741	0.814	0	0.814	362.8 MiB	28.2

rather than at their published settings, which is what makes the comparison internally fair but also means the numbers are not the numbers reported in the original papers; where a baseline's own configuration is materially different — the prompt methods' pool size and prompt length — we state both configurations, so that the comparison cannot be read as a deliberately weakened baseline. Training budgets are set per family rather than per method (Sec. 5.1), so the adapter-based baselines that Table 1 compares against FOLoRA share its budget; we report no baseline row at a budget we chose to disadvantage it. One gap in that protocol is worth closing rather than leaving it as an open question: the five-epoch adapter budget is justified by the behaviour of the *Fisher-weighted* methods, yet it is applied to the non-Fisher adapter baselines as well. We therefore trained Seq-LoRA, O-LoRA and InfLoRA for twenty epochs too, changing nothing else, and compared the two budgets on the three seeds they share (Table 8). *The longer budget makes all three baselines worse* — by 4.60 points for Seq-LoRA ($p = 0.091$), 10.40 for O-LoRA ($p = 0.003$) and 3.28 for InfLoRA ($p = 0.008$) — which is the signature of adapter over-fitting rather than of a budget that was starved: the five-epoch rows in Table 1 are the stronger of the two settings for every one of them. The asymmetry the protocol inherits therefore runs in the direction opposite to the one that would flatter our comparison, and we report both budgets rather than only the one that helps.

That leaves exactly one tabulated baseline that is *not* at the formulation its own paper specifies, and we report that exception and its resolution rather than a summary of the family. At inference we merge every per-task adapter for both O-LoRA and InfLoRA, which is the setting their papers

Table 8: Training budget for the three non-Fisher adapter baselines on Split CIFAR-100: five epochs per task (the budget FOLoRA and EWC-LoRA use) against twenty epochs per task, with everything else unchanged. Mean ± std over the three seeds both budgets share, so the five-epoch column is recomputed on those seeds and differs slightly from the full-seed-set rows of Table 1. Δ is the paired difference (twenty minus five) over seeds; p is the two-sided paired t-test, uncorrected, n=3.

Method	5 epochs	20 epochs	Δ	p
Seq-LoRA	68.98±1.86	64.39±3.43	−4.60	0.091
O-LoRA	67.60±0.75	57.21±0.95	−10.40	0.003
InfLoRA	73.12±0.55	69.84±0.14	−3.28	0.008

specify. On ImageNet-R that places both above the floor (53.72 for O-LoRA and 58.93 for InfLoRA, against 49.71); on CIFAR-100 it places InfLoRA above it (72.95 against 70.31) while the tabulated O-LoRA row remains below it (64.94). That row implements O-LoRA's orthogonal *initialization* without the training-time objective the original paper specifies — the formulation augments the loss with $\lambda_1 \sum_{i<t} \|A_t^\top A_i\|_F^2$ — so we ran that objective at two constraint strengths, λ_1=0.1 and λ_1=1.0 (n=5 on CIFAR-100, n=3 on ImageNet-R). It moves the CIFAR-100 mean from 64.94 to 71.21 ± 1.10 and 72.72 ± 0.90, and reaches 56.30 and 61.15 on ImageNet-R: every one of these means is above the floor. The margin is small at the weaker constraint — 0.9 points on CIFAR-100, not significant over the five seeds ($p = 0.14$), against 2.4 points at λ_1=1.0 ($p = 0.004$). Against these stronger configurations FOLoRA's advantage narrows from +12.92 to +7.10 and +5.60 on CIFAR-100, and from +13.20 to +10.59 and +5.74 on ImageNet-R, and it remains significant at both strengths after correction within the two-member

family these variants form — so we report both strengths for each benchmark rather than the one that suits us best, and the narrower figures are the margin we are entitled to claim. Both baselines are evaluated at the per-family budget Sec. 5.1 fixes, so no row of Table 1 is reported at a budget chosen to disadvantage it. What separates these configurations from FOLoRA is not the family but what the constraint acts on: an orthogonality penalty placed in the adapter input space bounds $A_t^\top A_i$ and leaves the merged update $\sum_i B_i A_i$ unconstrained, whereas FOLoRA's Fisher-weighted term penalizes the directions in which that update moves.

Baselines we did not run. Three adaptation methods a reader may expect to see here are absent, and we name the reason for each rather than leaving the omission implicit. Only RanPAC [70] is training-free, which is why it can be attached to a checkpoint we already have (Table 4); the other three are training procedures. EASE trains a distinct lightweight adapter per task and a prototype-complement classifier that synthesises old-class features [20], which places it in the per-task-module family this paper is designed to avoid. HiDe-Prompt trains task-specific prompt pools together with a task-identity-inference module [38], i.e. the prompt family already represented in Table 1 by L2P and CODA-Prompt. SLCA's slow learner fine-tunes the pre-trained backbone on the first task [73], so it is undefined on a frozen backbone and its parameter budget lies outside the one under which we compare. Reproducing any of the three would mean implementing its own training recipe at its own budget, and none of them would answer a question the RanPAC control leaves open.

The bound's slack. Finally, Sec. 4 reports that the forgetting bound is not tight at the operating point we probe: its remainder and its stationarity residual are the same order as its leading term, and the finite-sample Fisher estimate is not a calibrated surrogate for the measured curvature. The derivation is therefore used as a statement of *which* quantity to penalize, and the empirical support for the method is the ablation above rather than the bound. We prefer to state this explicitly, since the alternative — presenting a loose bound as if it were predictive — is the failure mode that a reader who re-derives Eq. (16) would find anyway.

What we do not establish. Seven things are left open, and we name them rather than letting the tables imply more. (i) *No advantage over a tuned diagonal EWC-LoRA.* At the tuned operating point of each benchmark the two are indistinguishable, and no regime we tested separates them there. The one condition in which the difference is significant is ImageNet-R at the pre-registered λ=100, which is the weakest point of EWC's grid on that benchmark: Sec. 5.2 reports the +2.85 ($p = 0.0002$) lead at that point and states explicitly that it is not a general advantage, because at $\lambda \in \{300, 1000\}$, both roughly 2.8 points higher, the difference is +0.05 and +0.04. The clearest test is a 50-task Split CIFAR-100 sequence (2 classes per task) that we ran as a generality check and did not tune for either method: FOLoRA's best point there is 74.25 ± 1.46 against 74.94 ± 0.69 for EWC-LoRA at λ=1000, a paired difference of -0.69 points ($p = 0.26$, n=5, uncorrected), with the ordering reversing across the λ grid exactly as it does at twenty tasks. Longer sequences therefore do not produce the separation a curvature-aware penalty might be expected to show. (ii) *No accuracy advantage over a well-tuned equal-weight*

penalty. The eigenvalue weighting reaches the ≈ 78 plateau at $\lambda = 3$ where the equal-weight arm needs $\lambda \in [10, 30]$, and it is steadier across seeds, but at their best-tuned points the two arms tie (Sec. 5.5), and at $\lambda = 30$ the equal-weight arm is the stronger of the two: the weighting narrows the usable window as well as advancing it. What we establish about the weighting is its derivation, its place in the family of penalties, and its operating point — not a higher ceiling. (iii) *Memory.* At k=64 the importance state is 32.5× EWC-LoRA's. Where that memory is not available, the diagonal baseline is the more practical method, and k=16 (8.5×) is the smallest point we tested; we make no claim that FOLoRA is preferable in that regime. (iv) *Evaluation-time data access.* The class means are recomputed from each task's training split at evaluation time (Sec. 5.1). No gradient is ever computed on a past task, but the assumption is real: a deployment that cannot retain past training data cannot reproduce these numbers. We measured what it costs to do without: under a strictly sequential protocol, in which prototypes are frozen at the end of the task that produced them, FOLoRA loses 8.5 points (69.43 ± 6.83 against 77.96), EWC-LoRA 14.0 (64.40 ± 5.69 against 78.42) and Seq-LoRA 26.8 (42.41 ± 6.17 against 69.17), over three seeds each. The ranking survives, the levels do not, and the numbers in Table 1 should be read as requiring the access. (v) *The classifier head.* The head is trained jointly and discarded at test time. Its gradients still shape the adapter, and whether training with no head at all — fitting the nearest-class-mean rule directly — would do better is untested here. (vi) *Task boundaries and backbone.* The method consumes an explicit task boundary, the evidence covers one backbone and two benchmarks, and LoRA is attached only to the MLP

down-projection. None of the three is varied, and we do not extrapolate past them. (vii) *No accuracy advantage over a tuned isotropic displacement penalty.* Replacing the Fisher-weighted orthogonality term by a plain penalty on the adapter displacement (L2-SP), scaled to the same RMS as the Fisher term and tuned over the same strengths, gives the same accuracy on CIFAR-100: 78.27 ± 0.38 against 78.37 ± 0.84 at $\lambda = 3$ ($\Delta = -0.10$, $p = 0.79$) and 77.77 ± 0.56 against 77.88 ± 0.58 at $\lambda = 10$ ($\Delta = -0.11$, $p = 0.52$), three common seeds each. What separates the two penalties in our experiments is the direction each protects, not the accuracy either reaches; whether the difference between them matters at a longer budget, a larger task count or a backbone with more interference is not established here.

6. Conclusion

We propose FOLoRA, a replay-free method for continual learning that mitigates catastrophic forgetting in low-rank adaptation while keeping the number of trainable parameters fixed as the number of tasks grows. FOLoRA maintains an accumulated second-order importance kernel for each adapted layer, forms an importance-weighted protected subspace from its top eigendirections, and penalizes alignment of the adapter's displacement with those directions in proportion to their importance. We derive a forgetting upper bound that decomposes into Fisher-weighted subspace overlaps and showed that FOLoRA's regularizer minimizes its dominant term, thereby providing a principled stability–plasticity trade-off and containing equal-weight orthogonalization as a special case. We also measure the slack of that bound instead of assuming it away: at the operating point we probe the remainder and

the stationarity residual are the same order as the quadratic term, so the bound is presented as a design principle for what to penalize rather than as a tight or numerically predictive characterization of forgetting. The empirical case for the regularizer rests on the λ=0 ablation, which removes the Fisher-weighted penalty while changing nothing else and returns the method to the level of unprotected sequential fine-tuning. Experiments on standard class-incremental benchmarks show that FOLoRA reaches an average accuracy on par with the strongest regularized baseline while using no replay buffer and no growth in the parameter count, and we quantify the importance-model memory it pays for representing that model as a rank-k subspace rather than EWC's diagonal one — 32.5× EWC's importance state at k=64 and 8.5× at k=16, which is the trade-off that makes k a memory–accuracy dial rather than a free parameter. When that memory is the binding constraint, k=16 at $\lambda \in \{3, 10\}$ is the setting to use: on CIFAR-100 it is within one point of the main configuration with no significant difference ($p \geq 0.27$), on ImageNet-R within 1.1 points with no significant difference either, and it costs 47.8 instead of 182.8 MiB. We also report the run-to-run reproducibility floor we measured (a single-run standard deviation of 0.63 points, i.e. 0.89 points for the difference of two runs at a fixed seed and a fixed configuration, from eleven repeated runs), and we state the resulting conclusions at the strength the data supports: parity with the strongest regularized baseline, rather than an improvement over it, and a significant advantage over the parameter-efficient prompt and subspace baselines.

A natural direction for future work is to extend FOLoRA to task-agnostic settings with distribution shift (domain-incremental learning) and to lan-

guage models, where the low-rank subspace geometry of different task families can be exploited more explicitly.

CRediT authorship contribution statement

Jiang Tao: Conceptualization, Methodology, Software, Validation, Formal analysis, Writing – original draft.

Declaration of competing interest

The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The datasets used in this study (CIFAR-100 and ImageNet-R) are publicly available standard continual-learning benchmarks. The source code implementing the proposed method, all baselines, the evaluation protocol and the scripts that reproduce every table and figure in this paper are openly available at `https://github.com/ml-universe/FOLoRA`. The repository ships the code, the configuration files, and the evaluation outputs under `reports/` from which the result tables are generated, so every reported number can be re-derived without retraining; it does not redistribute the benchmark datasets or the trained checkpoints, which are large and are regenerated by the training scripts.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author used Claude (Anthropic) in order to improve the language and readability of the manuscript. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

Acknowledgements

This research received no external funding.

References

[1] R. M. French, Catastrophic forgetting in connectionist networks, Trends in Cognitive Sciences 3 (4) (1999) 128–135. doi:10.1016/S1364-6613(99)01294-2.

[2] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, C. H. Lampert, icarl: Incremental classifier and representation learning, in: CVPR, 2017, pp. 5533–5542. doi:10.1109/CVPR.2017.587.

[3] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, et al., Overcoming catastrophic forgetting in neural networks, Proceedings of the National Academy of Sciences 114 (13) (2017) 3521–3526. doi:10.1073/pnas.1611835114.

[4] F. Zenke, B. Poole, S. Ganguli, Continual learning through synaptic intelligence, in: ICML, 2017.

[5] A. A. Rusu, N. C. Rabinowitz, G. Desjardins, et al., Progressive neural networks, arXiv preprint arXiv:1606.04671 (2016).
URL https://arxiv.org/abs/1606.04671

[6] Z. Wang, Z. Zhang, C.-Y. Lee, et al., Learning to prompt for continual learning, in: CVPR, 2022, pp. 139–149. doi:10.1109/CVPR52688.2022.00024.

[7] J. S. Smith, L. Karlinsky, V. Gutta, et al., Coda-prompt: Continual decomposed attention-based prompting for rehearsal-free continual learning, in: CVPR, 2023, pp. 11909–11919. doi:10.1109/CVPR52729.2023.01146.

[8] X. Wang, T. Chen, Q. Ge, et al., Orthogonal subspace learning for language model continual learning, in: Findings of the Association for Computational Linguistics: EMNLP 2023, 2023, pp. 10658–10671. doi:10.18653/v1/2023.findings-emnlp.715.

[9] D.-W. Zhou, H.-L. Sun, J. Ning, H.-J. Ye, D.-C. Zhan, Continual learning with pre-trained models: A survey, in: Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI), 2024, pp. 8363–8371. doi:10.24963/ijcai.2024/924.

[10] D.-W. Zhou, Z.-W. Cai, H.-J. Ye, D.-C. Zhan, Z. Liu, Revisiting class-incremental learning with pre-trained models: Generalizability and adaptivity are all you need, International Journal of Computer Vision 133 (3) (2025) 1012–1032. doi:10.1007/s11263-024-02218-0.

[11] E. N. Coleman, L. Quarantiello, Z. Liu, Q. Yang, S. Mukherjee, J. Hurtado, V. Lomonaco, Parameter-efficient continual fine-tuning: A survey, Neurocomputing 701 (2026) 134502. doi:10.1016/j.neucom.2026.134502.

[12] R. Razavi-Far, E. Hallaji, A. Fathalizadeh, M. Wu, M. Rostami, Towards self-adaptive learning: A comprehensive survey on continual learning under harsh conditions, Neurocomputing 698 (2026) 134136. doi:10.1016/j.neucom.2026.134136.

[13] J. Leo, J. Kalita, Survey of continuous deep learning methods and techniques used for incremental learning, Neurocomputing 582 (2024) 127545. doi:10.1016/j.neucom.2024.127545.

[14] E. J. Hu, Y. Shen, P. Wallis, et al., Lora: Low-rank adaptation of large language models, in: International Conference on Learning Representations (ICLR), 2022.

[15] Y. Wu, H. Piao, L.-K. Huang, R. Wang, W. Li, H. Pfister, D. Meng, K. Ma, Y. Wei, SD-LoRA: Scalable decoupled low-rank adaptation for class incremental learning, in: International Conference on Learning Representations (ICLR), 2025.
URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/92f43b1d33fae4aa1958f75317f0cec1-Abstract-Conference.html

[16] J. He, Z. Duan, F. Zhu, CL-LoRA: Continual low-rank adaptation for rehearsal-free class-incremental learning, in: Proceedings of the

IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2025, pp. 30534–30544. doi:10.1109/CVPR52734.2025.02843.

[17] H. Zhu, Y. Zhang, J. Dong, P. Koniusz, BiLoRA: Almost-orthogonal parameter spaces for continual learning, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2025, pp. 25613–25622. doi:10.1109/CVPR52734.2025.02385.

[18] X. Wei, G. Li, R. Marculescu, Online-LoRA: Task-free online continual learning via low rank adaptation, in: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), 2025, pp. 6634–6645. doi:10.1109/WACV61041.2025.00646.

[19] Y.-S. Liang, W.-J. Li, Inflora: Interference-free low-rank adaptation for continual learning, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024, pp. 23638–23647. doi:10.1109/CVPR52733.2024.02231.

[20] D.-W. Zhou, H.-L. Sun, H.-J. Ye, D.-C. Zhan, Expandable subspace ensemble for pre-trained model-based class-incremental learning, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024, pp. 23554–23564. doi:10.1109/CVPR52733.2024.02223.

[21] X. Wang, Z. Zhuang, Y. Zhang, PLAN: Proactive low-rank allocation for continual learning, in: Proceedings of the IEEE/CVF International

Conference on Computer Vision (ICCV), 2025, pp. 2909–2918. doi:10.1109/ICCV51701.2025.00279.

[22] J. Huang, L. Xu, Y. Liu, Z. Li, Reducing catastrophic forgetting via gradient-based task organization in class-incremental learning, Neurocomputing 706 (2026) 134964. doi:10.1016/j.neucom.2026.134964.

[23] R. Aljundi, F. Babiloni, M. Elhoseiny, et al., Memory aware synapses: Learning what (not) to forget, in: ECCV, 2018, pp. 144–161. doi:10.1007/978-3-030-01219-9_9.

[24] A. Mallya, S. Lazebnik, Packnet: Adding multiple tasks to a single network by iterative pruning, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 7765–7773. doi:10.1109/CVPR.2018.00810.

[25] A. Cossu, F. Giannini, G. Ziffer, A. Bernardo, A. Gepperth, E. Della Valle, B. Hammer, D. Bacciu, A practical guide to streaming continual learning, Neurocomputing 674 (2026) 132951. doi:10.1016/j.neucom.2026.132951.

[26] K. Li, C. Ding, J. Wan, S. Yu, Enhance the old representations' adaptability dynamically for exemplar-free continual learning, Neurocomputing 639 (2025) 130286. doi:10.1016/j.neucom.2025.130286.

[27] Q. Ferdinand, B. Clement, P. Papadakis, Q. Oliveau, G. Le Chenadec, Feature expansion and enhanced compression for class incremental learning, Neurocomputing 634 (2025) 129807. doi:10.1016/j.neucom.2025.129807.

[28] M. Liu, F. Ye, Learning diverse and adaptive representations for continual learning, Neurocomputing 664 (2026) 132112. doi:10.1016/j.neucom.2025.132112.

[29] A. Carta, A. Cossu, V. Lomonaco, D. Bacciu, J. van de Weijer, Projected latent distillation for data-agnostic consolidation in distributed continual learning, Neurocomputing 598 (2024) 127935. doi:10.1016/j.neucom.2024.127935.

[30] Y. Yang, J. Zhou, X. Ding, T. Huai, S. Liu, Q. Chen, Y. Xie, L. He, Recent advances of foundation language models-based continual learning: A survey, ACM Computing Surveys 57 (5) (2025) 1–38. doi:10.1145/3705725.

[31] Z. Wang, Z. Zhang, S. Ebrahimi, R. Sun, H. Zhang, C.-Y. Lee, X. Ren, G. Su, V. Perot, J. Dy, T. Pfister, Dualprompt: Complementary prompting for rehearsal-free continual learning, in: ECCV, 2022, pp. 631–648. doi:10.1007/978-3-031-19809-0_36.

[32] W. Fang, H. Che, F. Ren, J. Jiang, Q. Lao, Normality-preserving continual industrial anomaly detection via orthogonal LoRA banks, Neurocomputing 706 (2026) 135021. doi:10.1016/j.neucom.2026.135021.

[33] Q. Chen, J. Huang, W. Wen, Q. Li, R. Zhang, J. Qin, CAT: Continual adapter tuning for aspect sentiment classification, Neurocomputing 580 (2024) 127423. doi:10.1016/j.neucom.2024.127423.

[34] M. Li, P. Ye, L. Zhang, B. Bai, T. Chen, Adapter-X: A general

parameter-efficient fine-tuning framework for 2d and 3d vision, Neurocomputing 677 (2026) 133113. doi:10.1016/j.neucom.2026.133113.

[35] P. Wang, J. Wang, Z. Zhong, W. Bao, Y. Zhang, J. Chen, Enhancing long-term memory in federated class continual learning with lightweight adapters, Neurocomputing 656 (2025) 131459. doi:10.1016/j.neucom.2025.131459.

[36] D. Luo, K. Zheng, C. Wu, X. Wang, J. Wang, ERAT-DLoRA: Parameter-efficient tuning with enhanced range adaptation in time and depth aware dynamic LoRA, Neurocomputing 614 (2025) 128778. doi:10.1016/j.neucom.2024.128778.

[37] D.-W. Zhou, Z.-W. Cai, H.-J. Ye, L. Zhang, D.-C. Zhan, Dual consolidation for pre-trained model-based domain-incremental learning, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2025, pp. 20547–20557. doi:10.1109/CVPR52734.2025.01913.

[38] L. Wang, J. Xie, X. Zhang, M. Huang, H. Su, J. Zhu, Hierarchical decomposition of prompt-based continual learning: Rethinking obscured sub-optimality, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 36, 2023, pp. 69054–69076. doi:10.52202/075280-3022.

[39] L. Pan, Z. Lu, Y. Zheng, C. Yan, H. Wen, L. Tao, C. Xu, Task-adaptive continual learning of vision language models via prototype routing and

prompt, Neurocomputing 672 (2026) 132740. doi:10.1016/j.neucom.2026.132740.

[40] X. Xi, G. Cao, W. Cao, Y. Liu, Y. Li, H. Wang, H. Ren, Potential knowledge extraction network for class-incremental learning, Neurocomputing 616 (2025) 128923. doi:10.1016/j.neucom.2024.128923.

[41] C. Rebillard, J. Hurtado, A. Krutsylo, L. Passaro, V. Lomonaco, Continually learn to map visual concepts to language models in resource-constrained environments, Neurocomputing 652 (2025) 131013. doi:10.1016/j.neucom.2025.131013.

[42] H. Zhao, R. Liu, Y. Liu, Z. Zou, Y. Chen, Decoupled and distilled: Task-adaptive LoRA-teachers with ensemble knowledge transfer for few-shot class-incremental learning, Neurocomputing 708 (2027) 135200. doi:10.1016/j.neucom.2026.135200.

[43] S. Bayat Toksöz, G. Işık, G. Şahin, E. Akin, Parameter-efficient fine-tuning for photovoltaic cell defect classification: A systematic comparison of LoRA, QLoRA, and full fine-tuning on ConvNeXt-Tiny, Sensors 26 (12) (2026) 3659.

[44] S. Bayat Toksöz, G. Işık, TabEng-QLoRA: Criticality-aware tabular-to-text adaptation of large language models via saliency-guided quantized low-rank fine-tuning, Electronics 15 (8) (2026) 1728.

[45] M. Farajtabar, N. Azizan, A. Mott, A. Li, Orthogonal gradient descent for continual learning, in: AISTATS, 2020.

[46] G. Saha, I. Garg, K. Roy, Gradient projection memory for continual learning, in: ICLR, 2021.

[47] S. Wang, X. Li, J. Sun, Z. Xu, Training networks in null space of feature covariance for continual learning, in: CVPR, 2021, pp. 184–193. doi:10.1109/CVPR46437.2021.00025.

[48] M. A. Bennani, M. Sugiyama, Generalisation guarantees for continual learning with orthogonal gradient descent, arXiv preprint arXiv:1906.06195 (2019).
URL https://arxiv.org/abs/1906.06195

[49] Y. Lu, S. Zhang, D. Cheng, Y. Xing, N. Wang, P. Wang, Y. Zhang, Visual prompt tuning in null space for continual learning, in: Advances in Neural Information Processing Systems (NeurIPS), 2024, pp. 7878–7901, arXiv:2406.05658. doi:10.52202/079017-0253.
URL https://arxiv.org/abs/2406.05658

[50] E. Yang, L. Shen, Z. Wang, S. Liu, G. Guo, X. Wang, D. Tao, Revisiting flatness-aware optimization in continual learning with orthogonal gradient projection, IEEE Transactions on Pattern Analysis and Machine Intelligence 47 (5) (2025) 3895–3907. doi:10.1109/TPAMI.2025.3539019.

[51] C. Liu, Q. Liu, R. Li, X. Zhou, M. S. Kiran, X. Wang, A study on generalization of random weight network with flat loss, Neurocomputing 657 (2025) 131650. doi:10.1016/j.neucom.2025.131650.

[52] X. Wang, T. Liu, M. Zhou, Y. Zhang, Z. Yuan, K. Liu, P. Yang, Adapting to dissimilar tasks for continual learning via gradient norm regular-

isation, Neurocomputing 695 (2026) 134015. doi:10.1016/j.neucom.2026.134015.

[53] S. Adhikari, S. Agrawal, V. Sembium, FR-LoRA: Fisher regularized LoRA for multilingual continual learning, in: Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM), 2025, pp. 5559–5566. doi:10.1145/3746252.3761531.

[54] D. Han, S. Guo, FiLoRA: Parameter-efficient fine-tuning with Fisher information-guided low-rank adaptation, IEEE Signal Processing Letters 33 (2026) 604–608. doi:10.1109/LSP.2026.3652946.

[55] C. Zeng, S. Zhao, FI-LoRA: Dynamic and lightweight low-rank adaptation guided by Fisher information, IEEE Signal Processing Letters 33 (2026) 1260–1264. doi:10.1109/LSP.2026.3673210.

[56] Z.-Q. Feng, Y.-J. Lin, H.-Y. Kao, Learning in the Fisher subspace: A guided initialization for LoRA fine-tuning, ICML 2026 (Poster #64288); arXiv:2605.01046 (2026).
URL https://arxiv.org/abs/2605.01046

[57] Y. Zheng, Y. Zhang, J. van de Weijer, G. van de Ven, S. Du, X. Zhang, Z. Tian, Revisiting weight regularization for low-rank continual learning, in: Proceedings of the International Conference on Learning Representations (ICLR), 2026, arXiv:2602.17559.

[58] C. V. Nguyen, A. Achille, M. Lam, T. Hassner, V. Mahadevan, S. Soatto, Toward understanding catastrophic forgetting in continual learning,

arXiv preprint arXiv:1908.01091 (2019).
URL https://arxiv.org/abs/1908.01091

[59] I. Evron, E. Moroshko, R. Ward, N. Srebro, D. Soudry, How catastrophic can catastrophic forgetting be in linear networks?, in: Advances in Neural Information Processing Systems, 2022.

[60] R. Karakida, S. Akaho, Learning curves for continual learning in neural networks: Self-knowledge transfer and forgetting, in: ICLR, 2022.

[61] T. Doan, M. A. Bennani, B. Mazoure, G. Rabusseau, P. Alquier, A theoretical analysis of catastrophic forgetting through the ntk overlap matrix, in: AISTATS, 2021.

[62] J. Graldi, A. Breccia, G. Lanzillotta, T. Hofmann, L. Noci, The importance of being lazy: Scaling limits of continual learning, in: Proceedings of the 42nd International Conference on Machine Learning (ICML), Vol. 267 of Proceedings of Machine Learning Research, 2025.
URL https://proceedings.mlr.press/v267/graldi25a.html

[63] G. Zheng, P. Wang, S. Hu, Q. Zheng, L. Shen, Understanding the dynamics of forgetting and generalization in continual learning via the neural tangent kernel, in: International Conference on Learning Representations (ICLR), 2026.
URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/1f8638a87bddd34e090698f0bfaa9c02-Abstract-Conference.html

[64] R. Shuttleworth, J. Andreas, A. Torralba, P. Sharma, LoRA vs full fine-tuning: An illusion of equivalence, in: Advances in Neural Information Processing Systems (NeurIPS), 2025, pp. 193421–193456. doi:10.52202/085713-5808.

[65] G. Lai, D.-W. Zhou, X. Yang, H.-J. Ye, The lie of the average: How class incremental learning evaluation deceives you?, arXiv preprint arXiv:2509.22580 (2026).
URL https://arxiv.org/abs/2509.22580

[66] S. Cha, J. Kwak, D. Shim, H. Kim, M. Lee, H. Lee, T. Moon, Towards more diverse evaluation of class incremental learning: Representation learning perspective, in: Proceedings of the Conference on Lifelong Learning Agents (CoLLAs), Vol. 274 of Proceedings of Machine Learning Research, 2025, pp. 86–101.
URL https://proceedings.mlr.press/v274/cha25a.html

[67] G. Lee, Y. Oh, G. Jang, S. Lee, J. Song, S. Cha, Y. Yoo, ContinualMEGA: A large-scale benchmark for generalizable continual anomaly detection, Neurocomputing 700 (2026) 134460. doi:10.1016/j.neucom.2026.134460.

[68] J. Pomponi, A. Devoto, S. Scardapane, Class incremental learning with probability dampening and cascaded gated classifier, Neurocomputing 640 (2025) 130295. doi:10.1016/j.neucom.2025.130295.

[69] W. Zhang, P. Hu, Q. Li, J. Tan, Z. Tan, R. Wang, Boosting confidence in class-incremental learning via discriminative local feature enhance-

ment, Neurocomputing 660 (2026) 131829. doi:10.1016/j.neucom.2025.131829.

[70] M. D. McDonnell, D. Gong, A. Parvaneh, E. Abbasnejad, A. van den Hengel, RanPAC: Random projections and pre-trained models for continual learning, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 36, 2023, pp. 12022–12053. doi:10.52202/075280-0526.

[71] D. Castellana, A. Carta, CD-IMM: A Bayesian non-parametric classifier for continual learning, Neurocomputing 704 (2026) 134751. doi:10.1016/j.neucom.2026.134751.

[72] C. T. Tran, T. T. Nguyen, T. P. Nguyen, N. Thirion-Moreau, Geometry-controlled convex hull prototype framework for online task-free continual learning, Neurocomputing 701 (2026) 134619. doi:10.1016/j.neucom.2026.134619.

[73] G. Zhang, L. Wang, G. Kang, L. Chen, Y. Wei, SLCA: Slow learner with classifier alignment for continual learning on a pre-trained model, in: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2023, pp. 19148–19158.

[74] J. Martens, R. Grosse, Optimizing neural networks with kronecker-factored approximate curvature, in: Proceedings of the 32nd International Conference on Machine Learning (ICML), Vol. 37 of Proceedings of Machine Learning Research, 2015, pp. 2408–2417.

[75] F. Kunstner, L. Balles, P. Hennig, Limitations of the empirical fisher approximation for natural gradient descent, in: NeurIPS, 2019.

[76] A. A. Alhussan, E.-S. M. El-Kenawy, M. M. Eid, N. Khodadadi, Hybrid Al-Biruni and Puma optimization (BERPO) for boosting the classification of quality-of-service (QoS) in 5G networks, Journal of Network and Computer Applications 250 (2026) 104462. doi:10.1016/j.jnca.2026.104462.

[77] E.-S. M. El-kenawy, N. Khodadadi, S. Mirjalili, A. M. Zaki, A. Ibrahim, A. A. Alhussan, D. S. Khafaga, M. M. Eid, Glider snake optimizer (GSO): a nature-inspired metaheuristic algorithm for global and engineering optimization problems, Artificial Intelligence Review 59 (3) (2026) 91. doi:10.1007/s10462-026-11504-x.

[78] S. Barshandeh, N. Khodadadi, B. Abdollahzadeh, A. Mohammadzadeh, E.-S. M. El-Kenawy, M. M. Eid, K. M. Mosalam, Gray langurs optimizer: a multi-group bio-inspired optimization algorithm, Artificial Intelligence Review 59 (7) (2026) 166. doi:10.1007/s10462-026-11529-2.

[79] A. H. Alharbi, A. A. Alhussan, M. M. Eid, E.-S. M. El-Kenawy, Water membrane-based desalination modeling using a Ninja optimization-enhanced bidirectional long short-term memory with conceptually LLM-informed data preparation, Expert Systems with Applications 321 (2026) 132070. doi:10.1016/j.eswa.2026.132070.

[80] M. M. Eid, Renewable energy consumption forecasting using the Sword-

fish movement optimization algorithm (SMOA) for feature selection and hyperparameter tuning, Neural Computing and Applications 38 (11) (2026) 446. doi:10.1007/s00521-026-11964-w.

[81] D. Goswami, Y. Liu, B. Twardowski, J. van de Weijer, FeCAM: Exploiting the heterogeneity of class distributions in exemplar-free continual learning, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 36, 2023, pp. 6582–6595. doi:10.52202/075280-0288.

[82] S. Magistri, T. Trinci, A. Soutif-Cormerais, J. van de Weijer, A. D. Bagdanov, Elastic feature consolidation for cold start exemplar-free incremental learning, in: The Twelfth International Conference on Learning Representations (ICLR), 2024.
URL https://arxiv.org/abs/2402.03917

[83] S. Bayat Toksöz, G. Işık, SQ-LoRA: Memory-efficient language model compression through stable-rank-guided quantization for edge computing applications, Applied Sciences 16 (4) (2026) 2113.

Appendix A. Supporting statistics

Sec. 5.1 fixes the confirmatory family and states the correction policy. This appendix lists the individual comparisons behind that family, so that none of the verdicts has to be taken on trust or reconstructed from prose. Table A.9 *is* the confirmatory family: FOLoRA against each baseline of Table 1, on average accuracy, with the uncorrected and the Holm-corrected p side by side. Table A.10 collects the comparisons the body quotes in prose but no other table covers — the same-λ Fisher-weighting contrast, the k=16

sweep, and the two orthogonality strengths of O-LoRA — either because they are not against the main configuration or because their seed budgets differ from it. The two are kept apart rather than merged into one table because their correction conventions differ. Table A.9 is the confirmatory family and reports corrected p-values throughout. Table A.10 is exploratory: it is quoted uncorrected, with the single exception its caption states — the two O-LoRA orthogonality points form a two-member family per benchmark that the body quotes *with* its within-family correction, so that block's p_{Holm} column is filled in and the remaining rows leave it blank. A corrected p from one family should not be read against a corrected p from another.

Three conventions hold throughout. n is always the number of seeds *common* to the two arms of that row, never a shared budget, so rows with different n do not have the same power and their p-values should not be compared as if they did. Δ is FOLoRA minus the comparison, in accuracy points, so positive favours FOLoRA; on the gap metric reported in the body the sign convention is the opposite, which is one reason the two are tabulated separately. And a row whose n is below two carries no test at all: it is listed with p given as n/a rather than dropped, because the reason a point cannot support a claim is part of the record.

Appendix B. The probe behind Sec. 4.4

Sec. 4.4 states the conclusion of the bound-tightness probe and the main text uses only that conclusion. This appendix carries the protocol and the individual measurements, so that the claim can be checked rather than taken on trust.

Table A.9: The confirmatory family of Sec. 5.1: FOLoRA against each baseline of Table 1 on average accuracy, with both the uncorrected p and the Holm–Bonferroni corrected p_{Holm} that the paper's verdicts rest on. Conventions are stated above; the exploratory comparisons are in Table A.10.

Benchmark	Comparison	n	Δ	p	p_{Holm}
Confirmatory family: Table 1, average accuracy					
CIFAR-100	Seq-LoRA	10	+8.56	0.0000	0.0000
CIFAR-100	EWC-LoRA (lam=100)	10	+0.65	0.0734	0.2934
CIFAR-100	EWC-LoRA (lam=300)	10	-0.14	0.6440	0.9921
CIFAR-100	EWC-LoRA (lam=1000)	10	+0.21	0.4961	0.9921
CIFAR-100	EWC-LoRA (lam=3000)	10	+0.47	0.2507	0.7521
CIFAR-100	O-LoRA	10	+12.92	0.0000	0.0000
CIFAR-100	L2P	5	+4.67	0.0002	0.0013
CIFAR-100	CODA-Prompt	5	+3.61	0.0016	0.0080
CIFAR-100	InfLoRA	5	+5.37	0.0002	0.0014
ImageNet-R	Seq-LoRA	5	+10.34	0.0002	0.0011
ImageNet-R	EWC-LoRA (lam=100)	10	+2.85	0.0002	0.0011
ImageNet-R	EWC-LoRA (lam=300)	10	+0.05	0.9010	1.0000
ImageNet-R	EWC-LoRA (lam=1000)	10	+0.04	0.9235	1.0000
ImageNet-R	O-LoRA	5	+13.20	0.0000	0.0000
ImageNet-R	L2P	5	+7.45	0.0000	0.0000
ImageNet-R	CODA-Prompt	3	+7.58	0.0022	0.0066
ImageNet-R	InfLoRA	3	+7.96	0.0011	0.0046

Table A.10: The remaining comparisons the body quotes in prose, which no other table covers: variants of the main configuration and of one baseline, on both benchmarks. These are *exploratory* — chosen after seeing the results — so p is uncorrected and each entry's family is stated where its number is used in Sec. 5.2. The O-LoRA training-time block is the one exception: the body quotes those numbers with their within-family correction, given in the last column. Columns and conventions are those of Table A.9.

Benchmark	Comparison	n	Δ	p	p_{Holm}
Exploratory (uncorrected p)					
Fisher-weighted against equal-weighted, same λ					
CIFAR-100	λ=1	5	+3.83	0.0120	—
CIFAR-100	λ=3	10	+4.19	0.0000	—
CIFAR-100	λ=10	5	+0.38	0.6865	—
CIFAR-100	λ=30	5	-2.79	0.0039	—
k=16 protected directions against the k=64 main configuration					
CIFAR-100	λ=3	5	+0.17	0.7487	—
CIFAR-100	λ=10	5	+0.41	0.3832	—
CIFAR-100	λ=30	1	+0.36	n/a	—
CIFAR-100	λ=100	3	+3.02	0.1489	—
CIFAR-100	λ=300	10	+4.25	0.0000	—
CIFAR-100	λ=1000	3	+4.52	0.0561	—
O-LoRA orthogonality strength (p_{Holm}: two-member family)					
CIFAR-100	λ_1=0.1	5	+7.10	0.0000	0.0001
CIFAR-100	λ_1=1	5	+5.60	0.0002	0.0002
ImageNet-R	λ_1=0.1	3	+10.59	0.0172	0.0344
ImageNet-R	λ_1=1	3	+5.74	0.0174	0.0344

Protocol. All numbers below come from an in-training probe that, at each task boundary of a 20-task run, fits the expansion of the accumulated past-task loss along the *realized* adapter displacement, $\Delta\mathcal{L}(\gamma\,\delta\theta) = a\gamma + b\gamma^2 + c\gamma^3 + \mathcal{O}(\gamma^4)$, over $\gamma \in [0, 1.5]$. Measuring the nonlinearity along the realized displacement, rather than bounding L_3, is what makes the remainder checkable at all: L_3 is not recoverable from any stored checkpoint, whereas a, b and c are. We report a and c as ratios to $|b|$, so that "is this correction small?" is read directly off the number. The probe run is FOLoRA at λ=10, k=16, seed 0 on Split CIFAR-100, giving 19 usable boundaries. It is a single configuration, and we do not have the seed spread needed to say how much of what follows is run-to-run variation; the numbers should be read as one operating point, not as a distribution over the method.

(i) The remainder is not negligible. The ratio $|c|/|b|$ has median 0.163, minimum 0.010 and maximum 2.46, with 5 of the 19 boundaries above 0.30. Under the decision rule frozen before these numbers were computed (median ≤ 0.10: keep the negligibility claim; $0.10 <$ median ≤ 0.30: weaken it; > 0.30: drop it), this lands in the middle band, and the honest reading is the one the frozen rule prescribes: the cubic term is *of the same order as, but does not dominate*, the quadratic term. The tail is the part that matters — at the worst boundary the quantity the bound calls a remainder is 2.5× the quantity FOLoRA minimizes.

(ii) The stationarity residual is not small either. The ratio $|a|/|b|$ has median 0.99 and maximum 23.1. Assumption 1 is stated for the *unregularized* loss, so at a regularized endpoint the residual is $-\lambda\nabla\Omega$, and Remark 2 argued that it should vanish proportionally to the displacement. It does not vanish

fast enough to be dropped: at the realized displacement the linear term is comparable to the quadratic one, so the "quadratic plus corrections" reading of the expansion is not numerically available at this λ.

(iii) The Fisher estimate is not a calibrated surrogate for the curvature. The ratio between the term the bound prescribes, $\frac{1}{2}\delta\theta^\top \bar{F}\delta\theta$, and the quadratic coefficient b fitted from the same measurement has signed median 2.8×10^{-5} (median absolute ratio 6.6×10^{-5}): the finite-sample, rank-k, block-diagonal estimate of F_τ underestimates the measured curvature of the past loss along the realized displacement by four to five orders of magnitude. It is worth being precise about what this does and does not say. It does *not* contradict Theorem 1, which is stated for the exact F_τ; it says that Assumption 3 is far from innocuous at this operating point, which is precisely the regime the empirical-Fisher literature warns about [75]. It is also, in part, the intended behaviour seen from the other side: the displacement is close to orthogonal to the top-k Fisher subspace *by construction*, since suppressing that alignment is what the penalty does, so the Fisher-weighted quadratic form is small along it. Both readings agree on the operative point: the bound's leading term is not a quantitative predictor of forgetting in this setting.

(iv) Replication at the main configuration. The probe above is a single operating point, so we repeated the same measurement at the configuration the paper uses (λ=3, k=64) on both benchmarks and on every seed we had: three on Split CIFAR-100 and one on Split ImageNet-R, 19 usable boundaries each. The ratios are the same two quantities as above, with the same sign convention (a positive ratio means the correction has the sign of the quadratic term).

Run	median a/b	median c/b	$\lvert c/b\rvert > 0.3$	$\lvert c/b\rvert > 1$	median R^2
CIFAR-100, seed 0	-0.63	0.00	6/19	4/19	0.9979
CIFAR-100, seed 1	$+0.40$	-0.04	6/19	2/19	0.9999
CIFAR-100, seed 2	-0.55	0.16	8/19	3/19	0.9985
ImageNet-R, seed 0	-0.48	-0.01	3/19	0/19	0.9995

R^2 is that of the quadratic-only fit $\Delta\mathcal{L} = a\gamma + b\gamma^2$ over the same grid, i.e. how well the leading term alone describes the displacement before the corrections are added: it never falls below 0.978 at any boundary of any of these runs. The two facts therefore coexist — the second-order term is the right object to penalize, and the first-order cross term and the cubic remainder are not negligible at the operating point. That is the empirical justification for keeping them explicit in the statement of the bound and for treating ρ as an assumption rather than as a quantity the derivation controls.

Appendix C. Candidate-set curves

The two figures below re-score a *single* feature extraction of the terminal model of each run against candidate sets of increasing size, which is what the GAP column of Table 1 reduces to one number. They are placed here rather than in the body because the horizontal axis is easy to misread as a training trajectory; the body discusses their semantics where the axis is introduced.

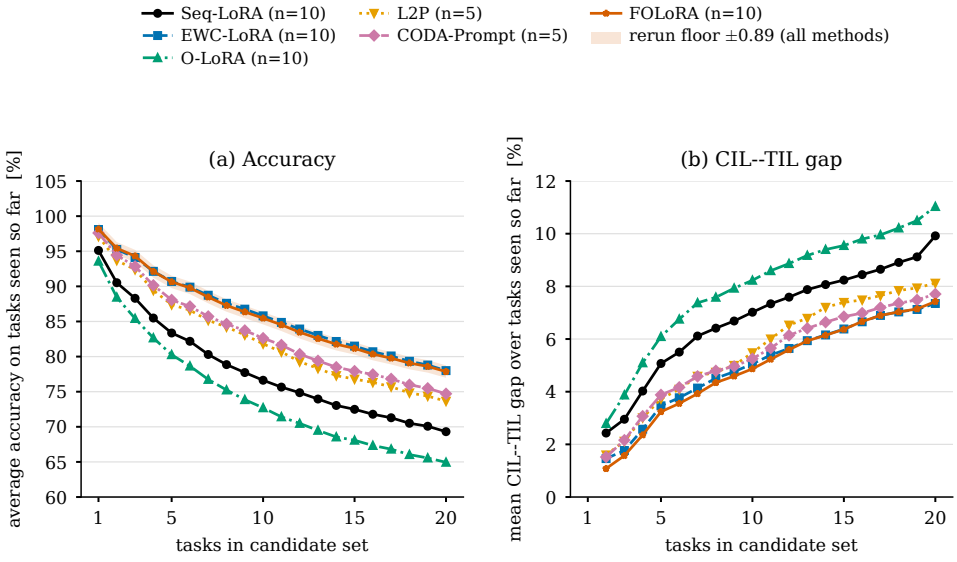


Figure C.1: Average accuracy over the candidate classes seen so far (left) and the mean CIL–TIL gap over those candidate sets (right) on Split CIFAR-100. Every point is a single feature extraction of the *terminal* model re-scored against a growing candidate set; the horizontal axis is the number of tasks in that candidate set (1 to 20, i.e. 5 to 100 candidate classes at 5 classes per task), not training time, so the leftmost point already contains the forgetting incurred over the whole sequence. Each curve is drawn at the configuration *and* the seed budget of its row in Table 1 — ten seeds for Seq-LoRA, EWC-LoRA, O-LoRA and FOLoRA, five for L2P and CODA-Prompt — so its right-hand endpoint reproduces that row's $\overline{\text{ACC}}$. Six of the eight rows of Table 1 are plotted; InfLoRA and the SimpleCIL floor are tabulated but not drawn here, so the legend and the table do not cover an identical row set. Those budgets differ between methods (Sec. 5.1), so the legend carries each curve's n and the figure implies no equal n. No smoothing or interpolation is applied — the markers are the per-task measurements. The shaded band is the run-to-run resolution of this measurement (±0.89 points, i.e. the standard deviation of the difference between two independent runs of one configuration; Sec. 5.6) — a property of the protocol rather than of any one method, and it applies to every curve shown. The vertical axis is bounded below at 60, i.e. 4.94 points under the lowest value plotted, rather than at zero; the full range is 60–105.

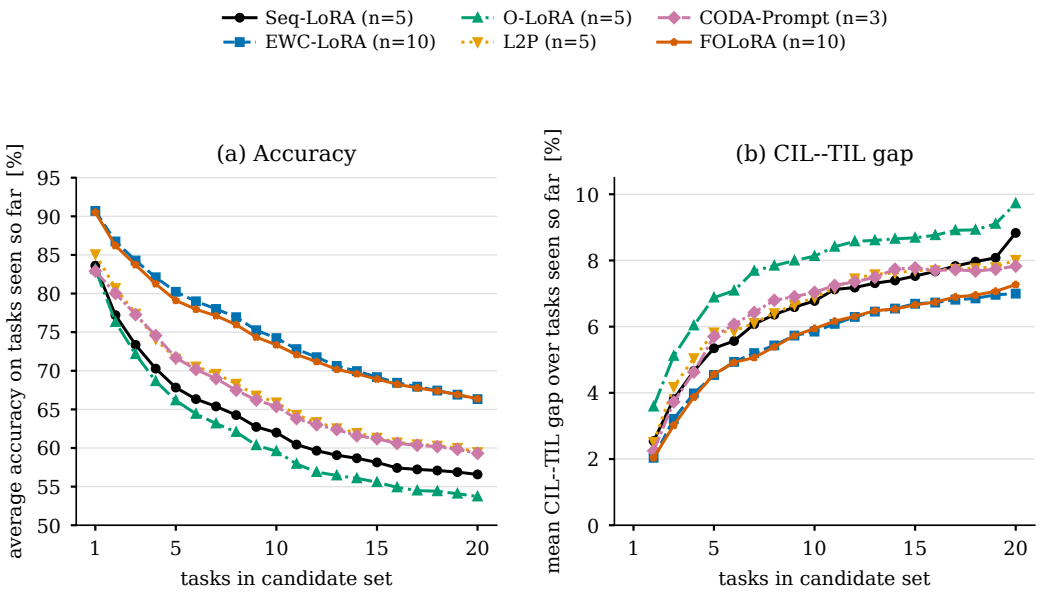


Figure C.2: Average accuracy over the candidate classes seen so far (left) and the mean CIL–TIL gap over those candidate sets (right) on Split ImageNet-R. As in Fig. C.1, each point is a single feature extraction of the terminal model re-scored against a growing candidate set — the number of tasks in it, 1 to 20, i.e. 10 to 200 candidate classes at 10 classes per task — not a training trajectory. Each curve is drawn at the configuration *and* the seed budget of its row in Table 1 — CODA-Prompt on three seeds, L2P, O-LoRA and Seq-LoRA on five, EWC-LoRA and FOLoRA on ten — so its right-hand endpoint reproduces that row's $\overline{\text{ACC}}$. As in Fig. C.1, six of the eight rows of Table 1 are plotted; InfLoRA and the SimpleCIL floor are tabulated but not drawn. Those budgets differ between methods (Sec. 5.1), so the legend carries each curve's n and the figure implies no equal n. No reproducibility-floor band is drawn, because that floor was measured on CIFAR-100 only and we do not transfer it to another dataset. The vertical axis is bounded below at 50, i.e. 3.72 points under the lowest value plotted.

Appendix D. Experimental environment

All experiments reported here were run on one machine, so the wall-clock figures of Table 7 are measured on the same device and under the same driver as each other; they are relative costs between methods, not absolute budgets. The GPU is an NVIDIA RTX 5060 Laptop with 8,151 MiB of memory (driver 592.01) under Windows 11, with Python 3.11.9, PyTorch 2.11.0 (CUDA 12.8), torchvision 0.26.0, NumPy 2.4.6, SciPy 1.17.1 and scikit-learn 1.9.0. Training is single-process, without data-parallel or mixed-precision accumulation. The backbone is the torchvision ViT-B/16 pre-trained on ImageNet-1k (`IMAGENET_1K_V1`), used frozen except where a baseline trains the backbone by construction. Both benchmarks are resized to 224×224 and normalised with the ImageNet statistics; the training pipeline adds a random horizontal flip, the test pipeline does not. CIFAR-100 is split into 20 tasks of five classes and ImageNet-R into 20 tasks of ten classes, with the class orders fixed by the seed reported alongside each result; every run writes its full configuration to a `config.json` in its own directory, so any number in this paper can be traced to the configuration that produced it. Each reported mean and standard deviation is over the seeds listed where the number is used; single-run figures are labelled as such.